UNIVERSITY OF PENNSYLVANIA ESE 5700

Project 1, Part 3: the MAC unit

Tyrone Marhguy • Victor Wanjohi • Due 10/27/2026

Cadence GPDK 18 nm $V_{DD} = 0.8$ V, nominal corner, 27°C $T_{CLK,\min}$ = **[TODO: ns]** 17 cycles START→DONE (predicted) area ≈ 292 μm^2 *(est.)*

Rubric map (handout, Part 3: "Report" list and 5-point grading)

Required item	Pts	Section	Status
Register cell with its characterization (setup, hold, t_{cq}, C_{CLK}, energy per clock)	1	§2	**Partial**: `tb_dff` and `tb_register_1b` run (functional); setup/hold/energy sweeps still open
Multiplier and adder schematics with sizes; logical-effort sizing of the critical path	1.5	§3	**Done**: schematics, LE path analysis (1.85 ns predicted vs 1.2 ns measured), sizing proposal, unit tests
State diagram, sequencer schematic, one-dot-product timing diagram	1.5 (with timing closure)	§4	**Partial**: state diagram, counter and controller verified (38/38, 60/60 vector checks); one-dot-product timing diagram missing
Timing closure: hand-computed critical path vs. measured $T_{CLK,\min}$; hold check	(above)	§5	**Missing**: critical path named; numbers need §2 and `tb_top`
Verification waveforms (Test A) and pass/fail table	1 (with metrics and report quality)	§6	**Done**: A, B, C pass at $T = 10$ ns; Test A ACC/DONE waveform
Metrics table: FOM$_{MAC}$, D, V_{DD}; energy per MAC with and without memory; where the energy goes	(above)	§7	**Partial**: area estimated; delay, energy, FOM missing
Short paragraph: what limited the design		§8	**Partial**: first draft; final numbers missing
Top-level block diagram and schematic; team members and contributions		§1	**Partial**: `top` schematic in; block diagram and contributions missing

Update the status colors (red missing, yellow partial, green done) as sections fill in. All simulations are schematic level (no extracted parasitics): bitline capacitance $C_{BL} = 3.5$ fF per bitline (`cbw`, as in Part 2), and each block output under test drives one `inv_2x`. Strategy from the handout: treat the SRAM as a black box with the Part 2 interface and t_{access}; register first, then the multiplier (exhaustive, all 256 pairs), then the accumulator, then the sequencer; simulate the full unit only when every piece passes alone.

1. Team members and contributions

Member	Contribution
Victor Wanjohi	**[TODO: fill in]**
Tyrone Marhguy	**[TODO: fill in]**

Summary. Two copies of the Part 2 macro hold the weights W and activations X; a static-CMOS array multiplier, a 12-bit ripple-carry accumulator, a 4-bit address counter and a two-bit controller (all registers are transmission-gate master-slave flip-flops) compute $\mathrm{ACC} = \sum_{i=0}^{15} W[i]\,X[i]$. The read is pipelined against the multiply-accumulate (read in cycle n, accumulate in $n+1$), so a dot product takes 17 cycles from the START edge to DONE (cycle-level model of the netlist). The multiplier is correct for all 256 input pairs and its worst case is about 1.2 ns (1.14–1.24 ns), so it, not the 488 ps SRAM read, sets the clock period. Area $\approx 292\ \mu\mathrm{m}^2$ *(est.)*. **[TODO: $T_{CLK,\min}$, Tests A/B/C result, energy.]**

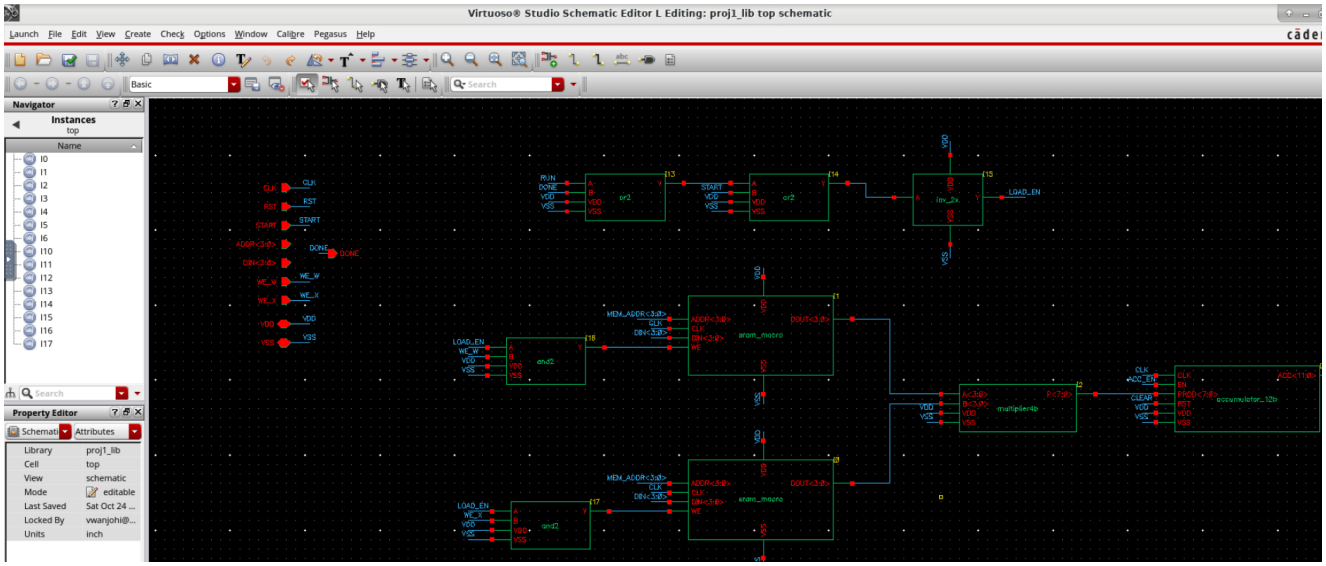


Figure 1: `top`, left half: the two `sram_macro` instances (I1 weights, written when LOAD_EN·WE_W; I0 activations, LOAD_EN·WE_X), sharing MEM_ADDR, CLK and DIN; their DOUT buses feed `multiplier4b` (I2), whose product feeds `accumulator_12b` (I3). Top: LOAD_EN = $\overline{\mathrm{RUN} + \mathrm{DONE} + \mathrm{START}}$ (two `or2`, `inv_2x`).

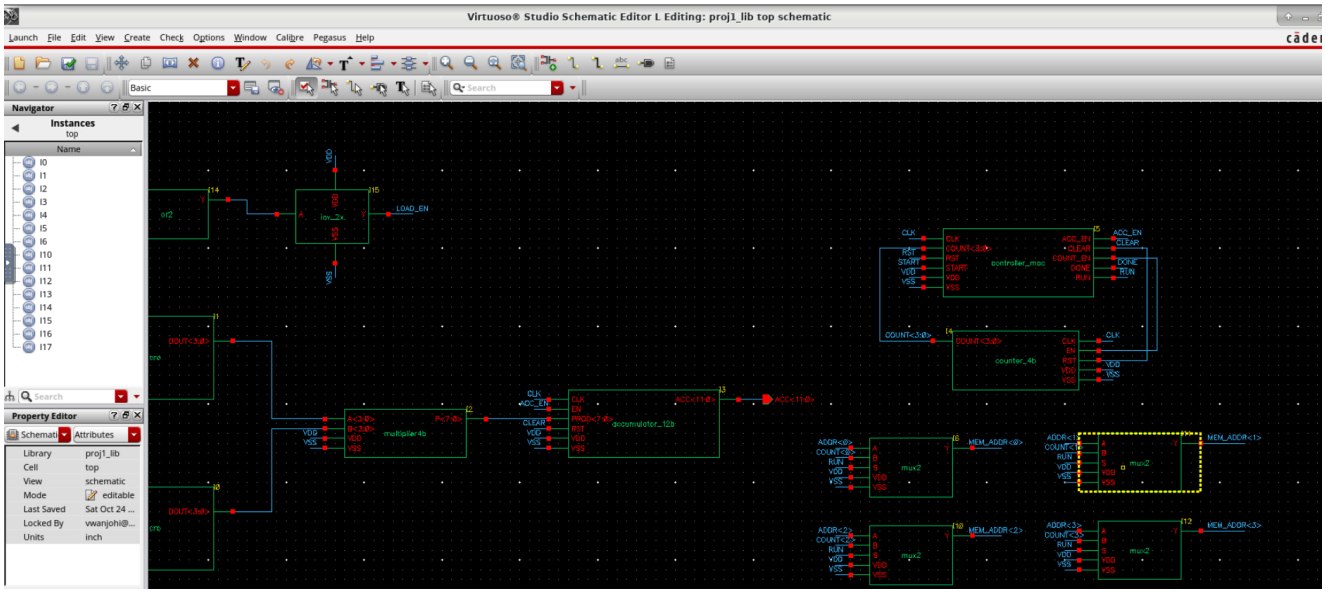


Figure 2: `top`, right half: `controller_mac` (I5) and `counter_4b` (I4) in a loop (COUNT_EN, CLEAR, COUNT⟨3:0⟩), and four `mux2` that send COUNT to MEM_ADDR while RUN = 1 and the external ADDR otherwise.

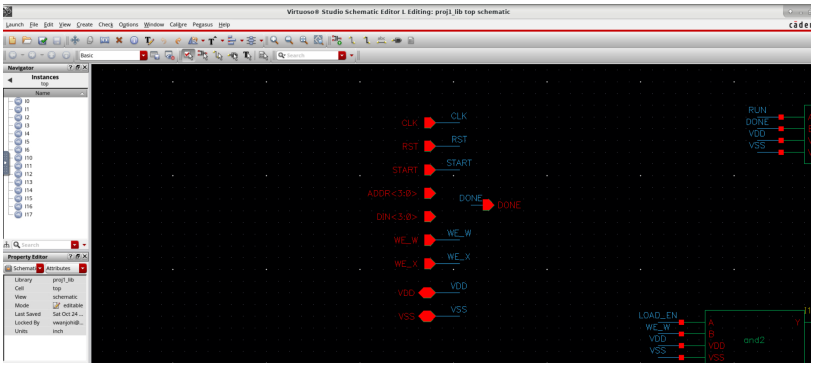


Figure 3: `top` ports: CLK, RST, START, ADDR⟨3:0⟩, DIN⟨3:0⟩, WE_W, WE_X in; DONE and ACC⟨11:0⟩ out; VDD, VSS.

Block inventory and test benches.

Block	Built from	Test bench	Status
Register bit (DFF)	latch pair, T-gates	`tb_dff`, `tb_register_1b`	pass at 2 ns; no t_{su}/t_h swe
4×4 multiplier	AND array, adders	`tb_multiplier4b`	256/256 correct; worst cas
12-bit adder / acc.	`fulladder_12b`, `register_12b`	`tb_fulladder_12b`, `tb_accumulator_12b`	adder sums match; acc. re
4-bit counter	4 DFFs, toggle chain	`tb_counter4b`	38/38 vector checks pass
Controller (idle/compute/done)	4 registers, gates	`tb_controller_mac`	60/60 vector checks pass;
MAC top	all of the above and 2 macros	`tb_top`	Tests A, B, C pass at $T =$

2. Sequential elements (3.1)

Rubric (3.1): register bit at transistor level, used for accumulator, counter, controller state, DONE; setup, hold, clock-to-Q with the simulation that determines each; clock-input capacitance; energy per clock; style justified.

Style and justification. Positive-edge master-slave flip-flop built from two static transmission-gate latches (`dlatch`): the master is transparent while CLK = 0, the slave while CLK = 1, so D is captured on the rising edge. Each latch has an input `tg`, a storage node driven by two `inv_2x`, a hold `tg` that closes the loop from Q, and its own clock inverter; the flip-flop adds one more `inv_2x` to make the master's clock (100 fins: `tg` 2+2 fins, `inv_2x` 8/4 fins). It was chosen because it reuses the verified `tg` and inverter cells, is fully static (holds with the clock stopped, unlike a dynamic C^{2}MOS latch), and needs only the single external clock. Its cost is the local clock inverters (clock load, and a small skew between the two phases, which hold must cover). `register_1b` adds a `mux2` load enable ($EN = 1$ loads D, $EN = 0$ recirculates Q) and an `and2` synchronous reset (152 fins). **[TODO: Alternative considered (e.g. C^{2}MOS or pulsed latch) and why rejected, with numbers once `tb_dff` has run.]**

Design note. The first `dlatch` fed the node after one inversion back through the hold gate, an inverting loop that cannot store a value; the hold `tg` now returns Q (two inversions) to the storage node and conducts while CLK = 0 (fixed 10/25, before any register simulation).

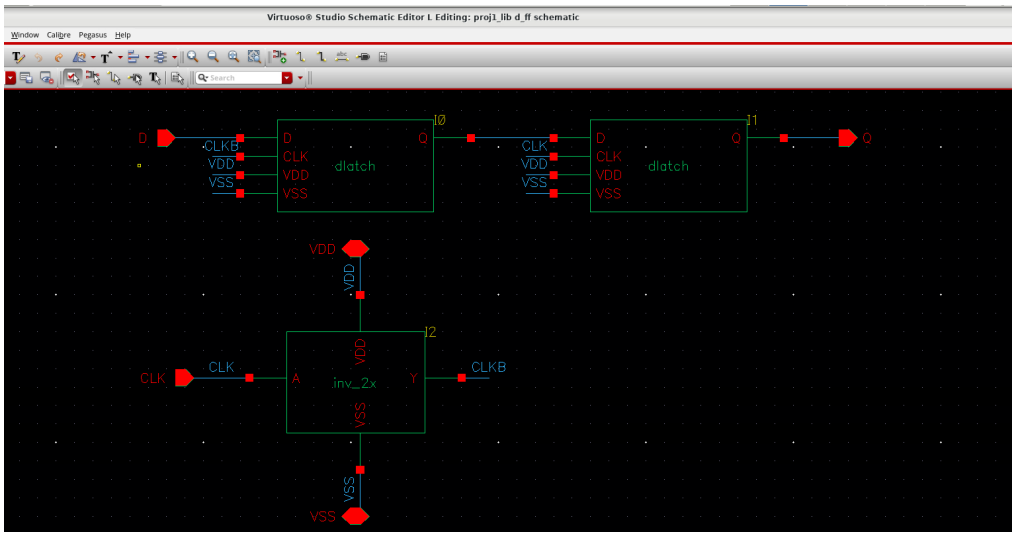


Figure 4: `d_ff`: master `dlatch` I0 clocked by CLKB (transparent while CLK = 0), slave I1 clocked by CLK, and the clock inverter I2 (`inv_2x`, 8/4 fins). Each `dlatch`: input and hold `tg` (2+2 fins), two `inv_2x` in the loop and one for its local clock; 100 fins in total.

Test bench (`tb_dff`). DUT is `d_ff`. CLK and D are `vpulse` sources, $0 \to 0.8$ V, $t_r = t_f = 20$ ps; VDD is 0.8 V; Q is loaded by 1 fF. CLK period is 2 ns (rising edges at $1, 3, 5, \ldots$ ns). D is an 8 ns period, 4 ns-high pulse (edges at $2, 6, 10, 14$ ns), so each data value is stable for a full cycle before the capturing edge.

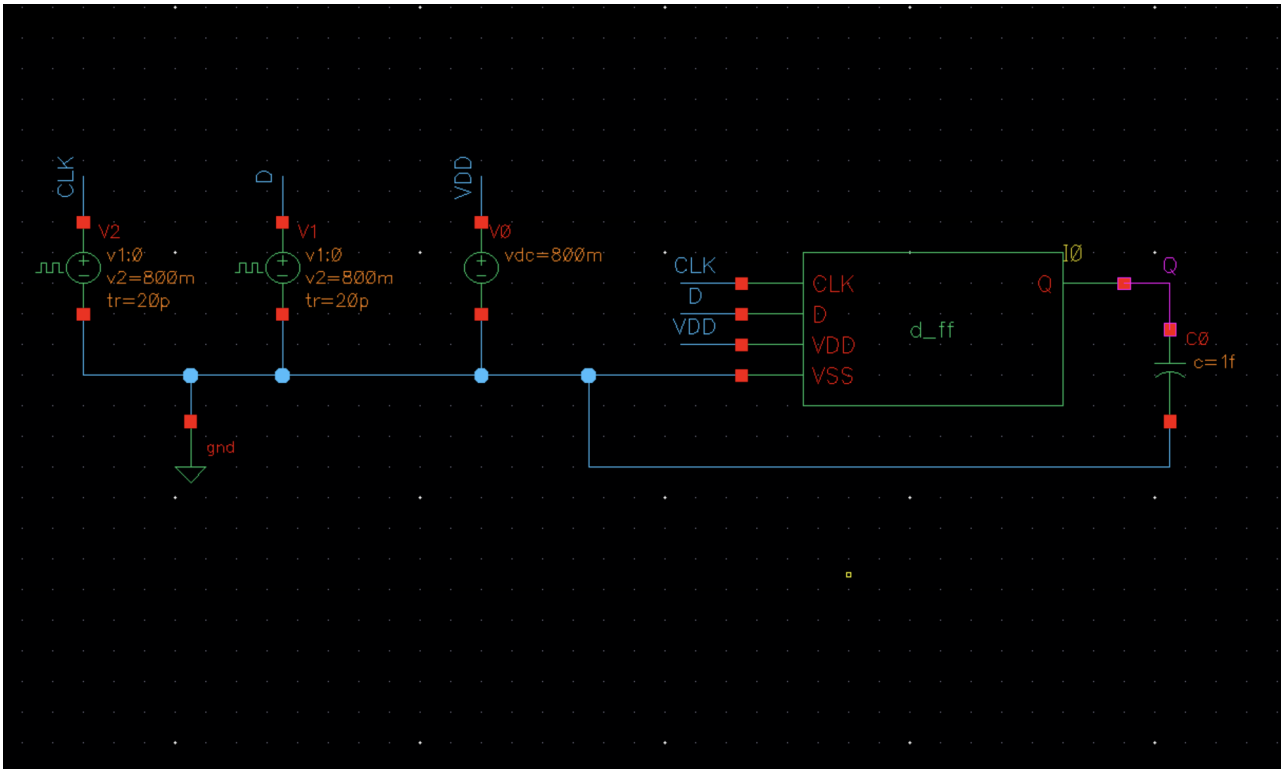


Figure 5: `tb_dff`: `d_ff` driven by CLK (V2) and D (V1), $V_{DD} = 0.8$ V (V0), $C_Q = 1$ fF, 20 ps edges.

The 0–15 ns transient is a functional check of the hold-loop fix, not a setup/hold bisection. Q powers up near mid-rail (≈ 420 mV), is forced to 0 on the first rising CLK ($D = 0$), then tracks D on every later rising edge: D rises at 2 ns $\to$ Q rises at 3 ns; D falls at 6 ns $\to$ Q falls at 7 ns; D rises at 10 ns $\to$ Q rises at 11 ns. Clock-to-Q, read from the close-up at the 3 ns and 7 ns edges, is ≈ 100 ps (50% CLK to 50% Q) for both rise and fall. Setup and hold walls, C_{CLK} and energy per clock are not in this run: D never approaches the CLK edge (nominal slack 1 ns).

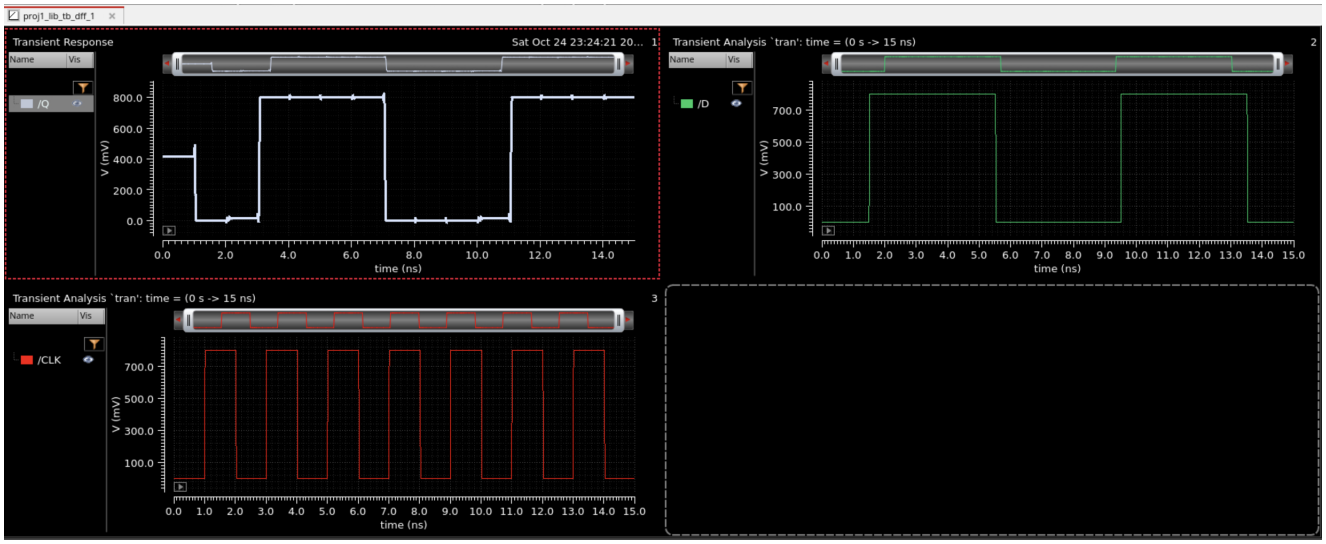


Figure 6: `tb_dff` transient, 0–15 ns: Q (left), D (right), CLK (bottom). Positive-edge capture; $T_{CLK} = 2$ ns.

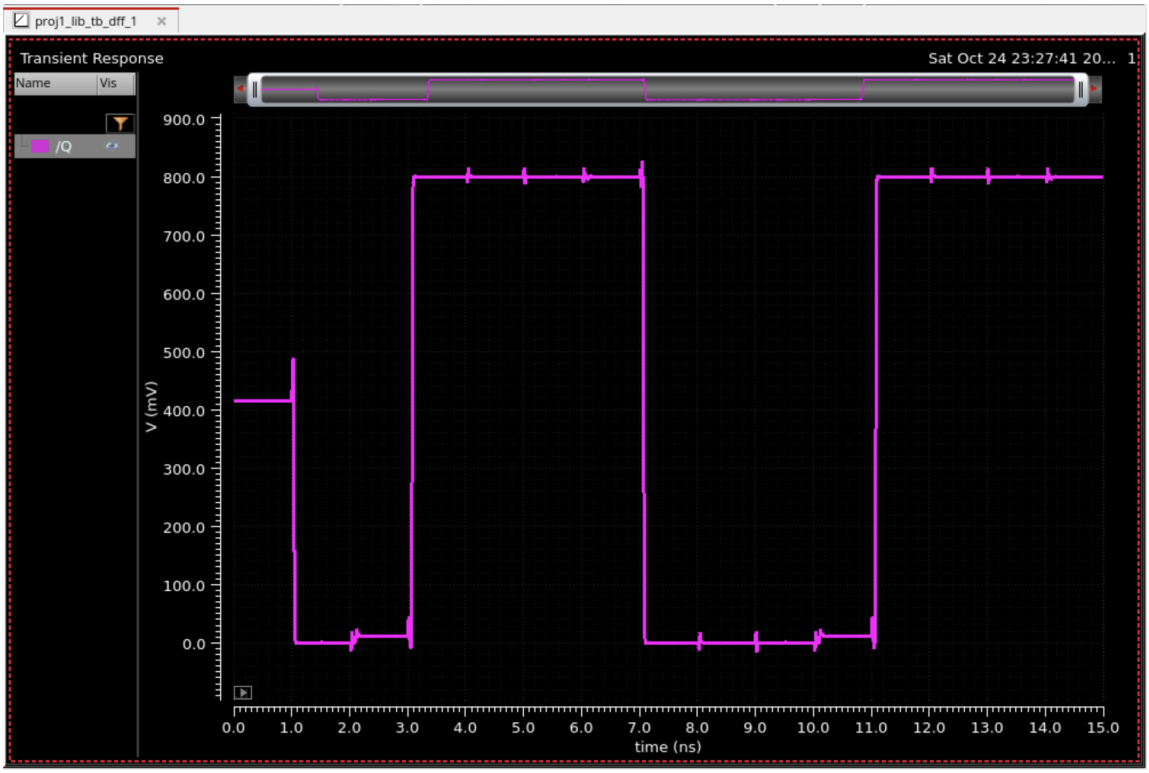


Figure 7: Q close-up: capture at 3.1 ns (rise) and 7.1 ns (fall). $t_{cq} \approx 100$ ps. Glitches at integer nanoseconds are clock feedthrough on the slave.

`register_1b` wraps the flip-flop with a `mux2` load enable and a synchronous reset. `tb_register_1b` drives CLK, D, EN and RST from four `vpulse` sources (0.8 V, 20 ps edges).

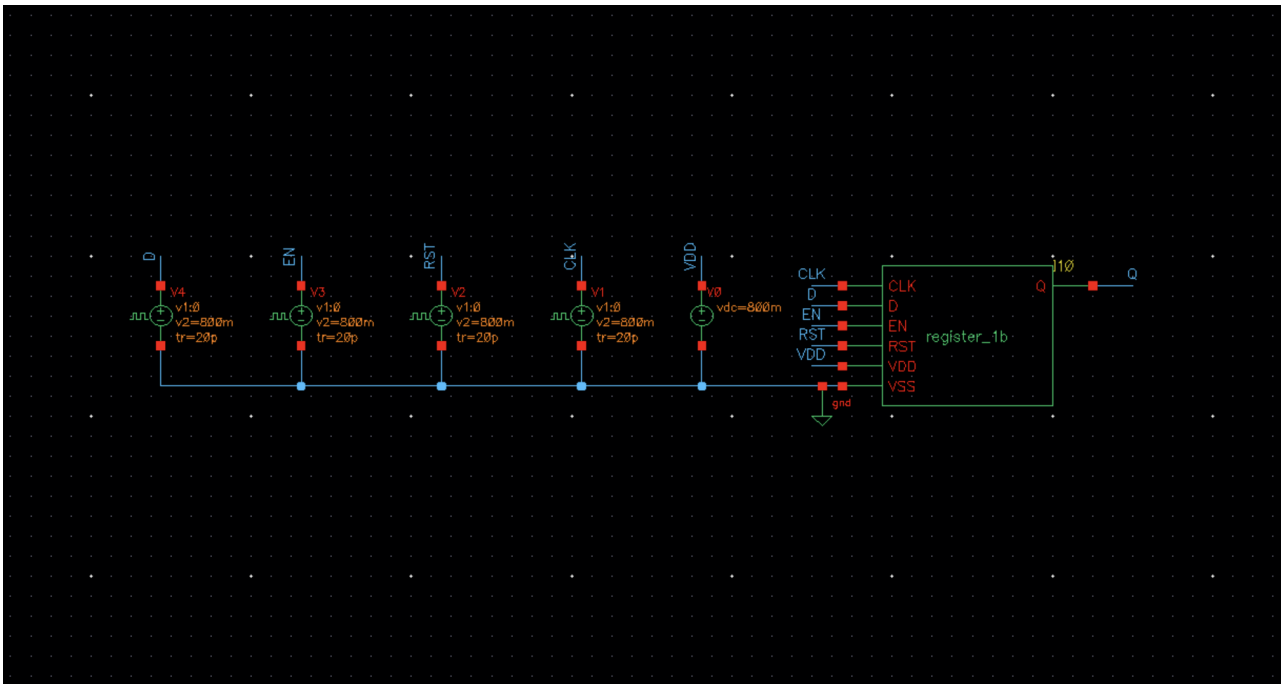


Figure 8: `tb_register_1b`: `register_1b` with CLK (V1), RST (V2), EN (V3), D (V4), $V_{DD} = 0.8$ V.

RST is active-low: it sits high except for a low pulse near 10 ns. EN is high 0–6.5 ns (load), low 6.5–12 ns (hold / recirculate), high 12–16 ns (load). Q follows D while EN = 1, holds 1 through the EN = 0 window until the RST pulse, then clears on the next rising CLK (≈ 11 ns). After RST releases, EN returns high and Q loads again (13 ns rise, 15 ns fall).



Figure 9: `tb_register_1b`, 0–20 ns: VDD, RST, mux output, Q, internal AND, EN. Hold while EN = 0; Q clears on the CLK after RST goes low.

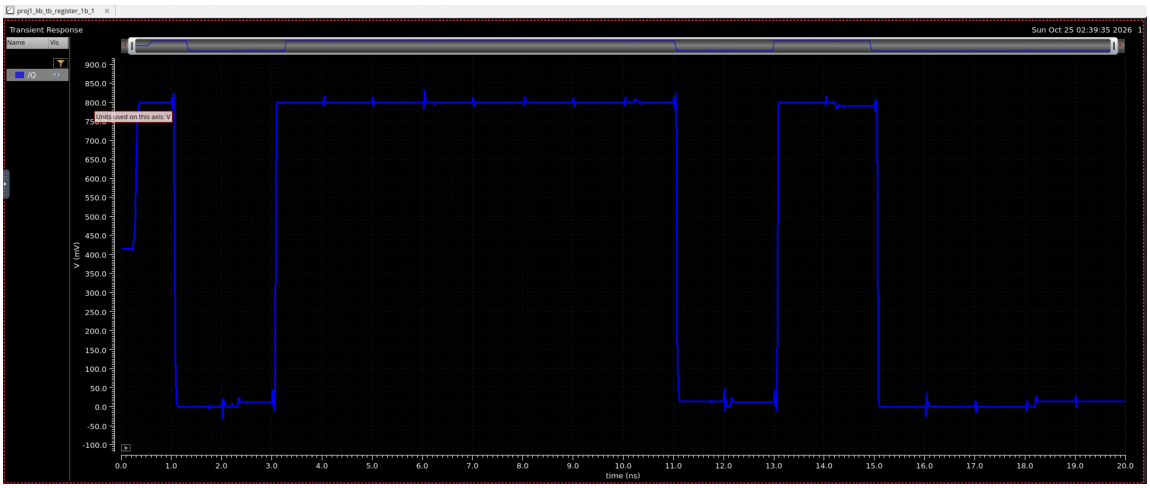


Figure 10: Q close-up: load (3.1 ns), hold through EN = 0, reset at 11.1 ns, reload at 13.1 ns, D = 0 captured at 15.1 ns.

`register_12b` is twelve **`register_1b`** in parallel (accumulator word). **`tb_register_12b`** drives the 12-bit D bus, CLK, EN and RST at $V_{DD} = 0.8$ V.

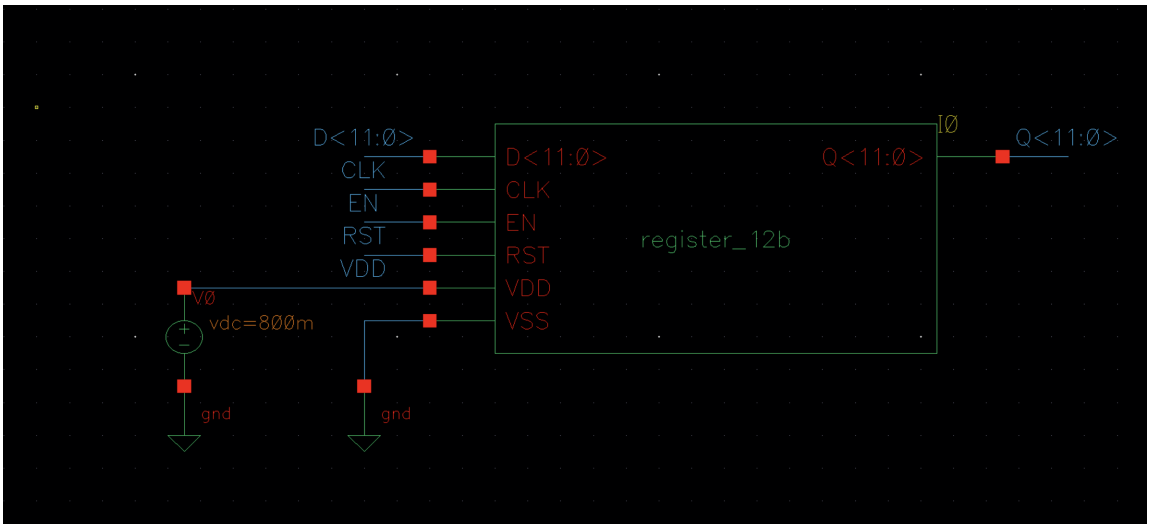


Figure 11: **`tb_register_12b`**: **`register_12b`** with D⟨11:0⟩, CLK, EN, RST; Q⟨11:0⟩ out.

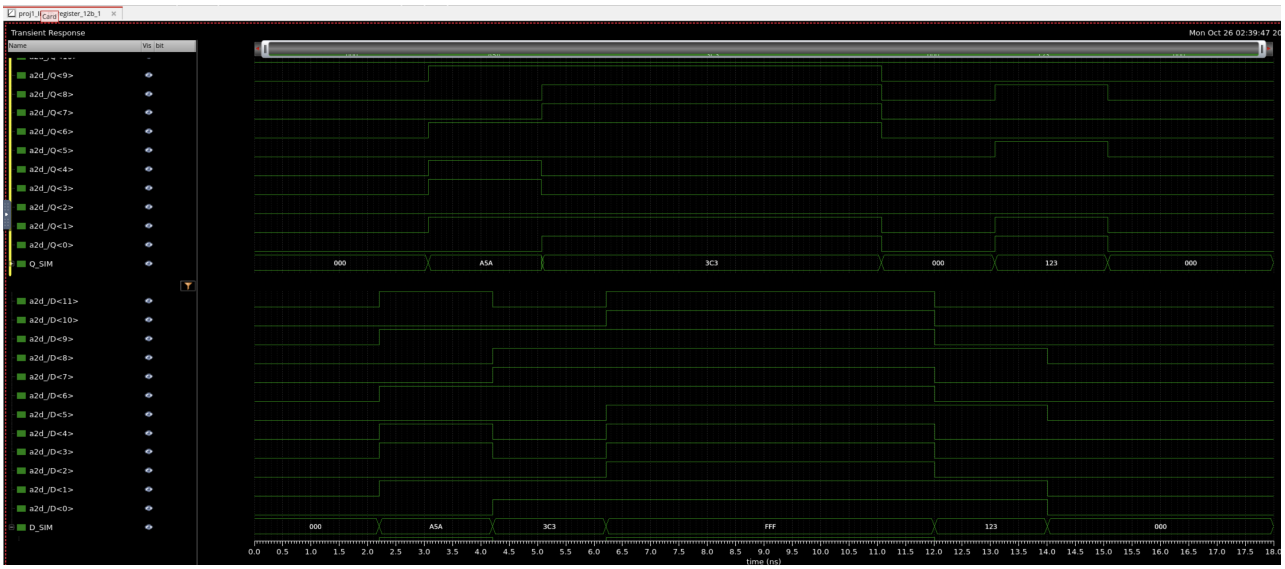


Figure 12: `tb_register_12b`: Q follows D on the rising CLK (000 →A5A→3C3→ 123). D =`FFF` while EN/RST holds Q, then Q returns to 000.

Quantity	Value	Determined by
Setup time t_{su} (D 0→1 / 1→0)	**[TODO: ps]**	D-to-CLK offset sweep; not in the functional run (slack 1 ns)
Hold time t_h	**[TODO: ps]**	same criterion, D changing after the edge
Clock-to-Q t_{cq} (rise / fall)	≈ 100 ps	CLK 50% to Q 50% on the 3 ns and 7 ns edges, $C_Q = 1$ fF
Clock-input capacitance C_{CLK}	**[TODO: fF]**	charge per cycle from CLK source / V_{DD}
Energy per clock	**[TODO: fJ]**	integral of V_{DD} current over one cycle, D toggling / not toggling

3. Datapath (3.2)

Rubric (3.2): 4×4 unsigned multiplier and 12-bit accumulator adder; any architecture justified; critical path sized with logical effort before simulating.

Leaf cells. Static CMOS throughout, $L = 18$ nm: `tg` (2+2 fins), `mux2` (two `tg` and an `inv_2x`, $Y = S\,?\,B : A$), `xor2` (complementary static gate with input inverters, 44 fins), `and2` (`nand2_u`+`inv_2x`), `or2` (NOR2+`inv_2x`). The full adder is $S = (A \oplus B) \oplus C_{in}$, $C_{out} = C_{in}(A \oplus B) + AB$ from two `xor2`, two `and2` and an `or2` (148 fins).

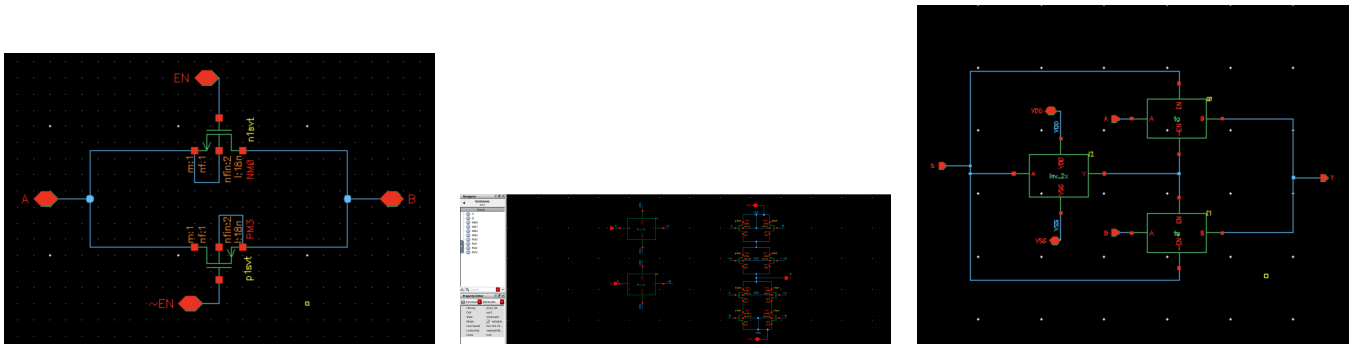


Figure 13: Leaf cells: `tg` (nfin 2 each), `xor2`, `mux2`. The `xor2` view is the corrected one (two series pull-down stacks). **[TODO: Re-capture `mux2` after the select swap.]**

Multiplier. Array multiplier: 16 AND gates form the partial products $a_i b_j$ (four `and2x4`, one per row j), and three 4-bit ripple-carry adders (`fulladder_4b`, 12 full adders) sum the rows, each row shifted by one bit; $P_0 = a_0 b_0$, $P_{1..3}$

from the low bit of each adder row, $P_{4..7}$ from the last row and its carry. It reuses one verified full-adder cell in a regular array, which made it quick to build and debug; the cost is speed, because the longest path ripples through all three adder rows into the last row's carry (P_6, P_7). A carry-save (Wallace or Dadda) tree would shorten the path to two full-adder levels plus one final carry-propagate adder, for the same 12 full adders but irregular wiring. 2096 fins.

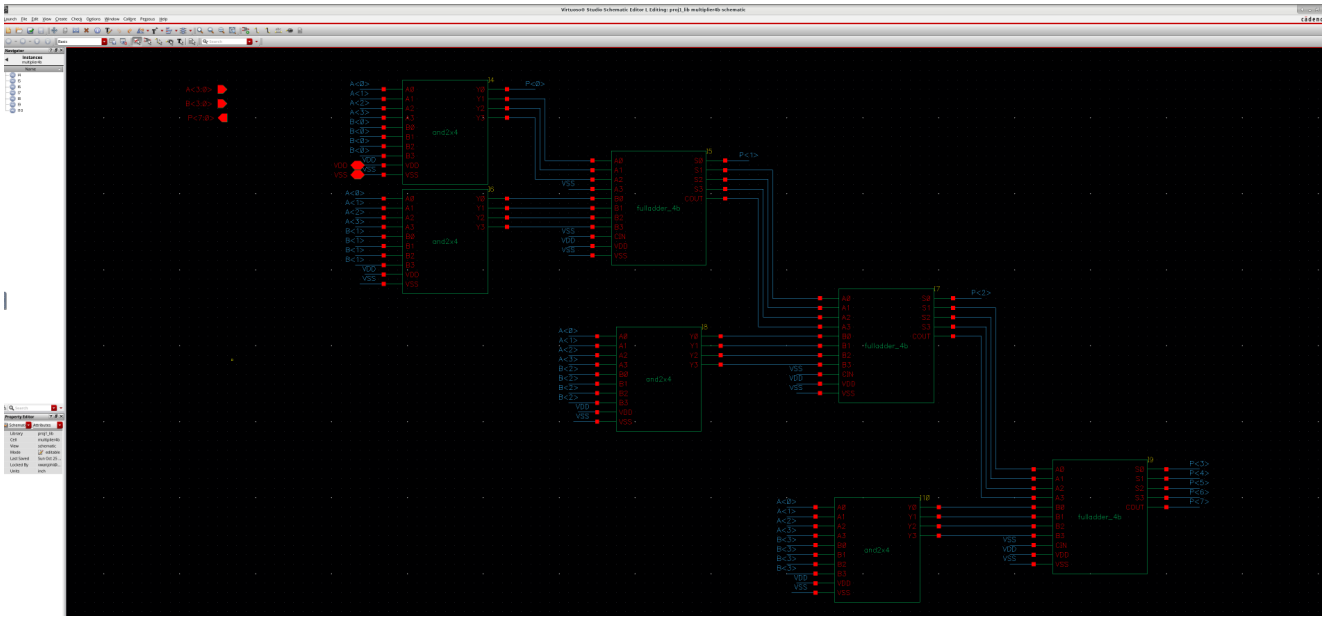


Figure 14: `multiplier4b` (corrected): partial-product rows I4, I6, I8, I10 (`and2x4`, row j gets $A\langle 0..3\rangle$ and B_j on all four B pins) and ripple rows I5, I7, I9 (`fulladder_4b`, $C_{in} = 0$); each row adds the previous row's upper sum bits and carry to the next partial product. Sizes ($L = 18$ nm): `and2` = `nand2_u` (PMOS 2‖2, NMOS 2+2 series fins) + `inv_2x` (8/4); full adder: two `xor2` (NMOS 2/4, PMOS 2 fins, input `inv_2x`), two `and2`, one `or2`.

Accumulator adder. 12-bit ripple-carry adder of the same full adder (`fulladder_12b`): $A = \mathrm{ACC}\langle 11{:}0\rangle$, $B = \{0000, P\langle 7{:}0\rangle\}$, $C_{in} = 0$; the sum goes to `register_12b` (12 `register_1b`, $EN = \mathrm{ACC_EN}$, synchronous reset on CLEAR). Twelve bits are enough: the largest result is $16 \times 15 \times 15 = 3600 < 4096$. Only the low 8 bits receive the product, so the upper 4 stages only propagate the carry. 3600 fins for the accumulator (1776 adder, 1824 register).

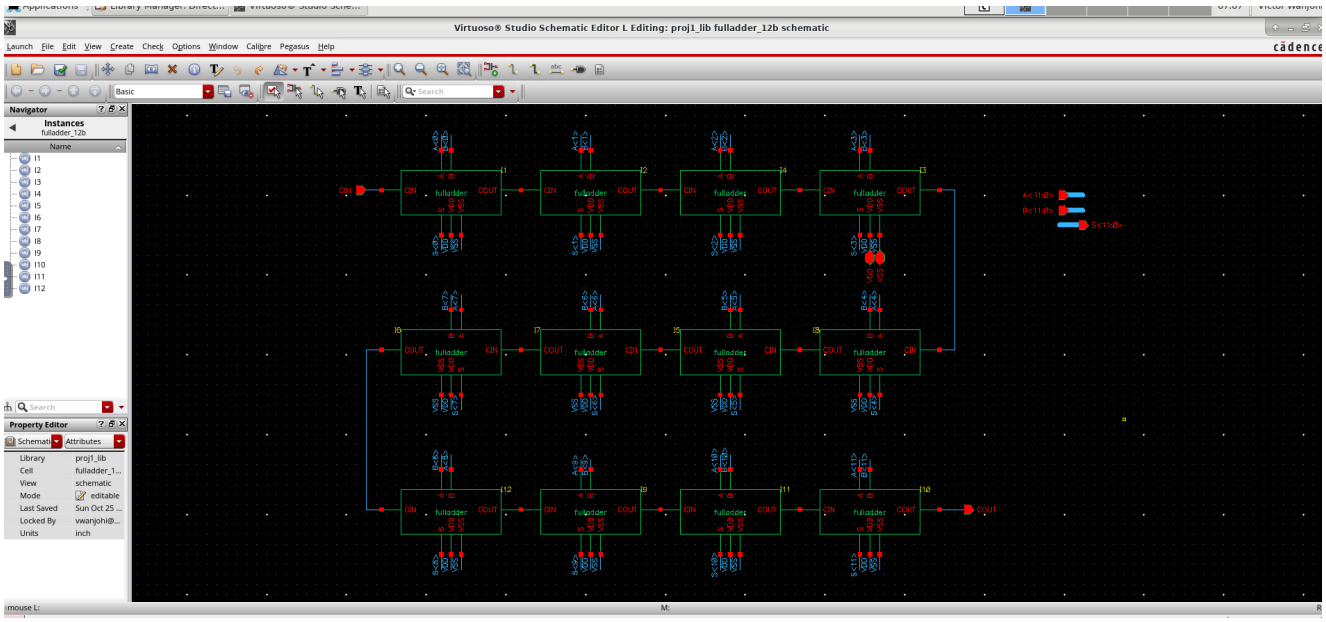


Figure 15: `fulladder_12b`: twelve `fulladder` cells in a ripple chain (I1 bit 0 with C_{in} to I10 bit 11 with C_{out}), drawn as three rows of four; each cell is 148 fins (two `xor2`, two `and2`, one `or2`).

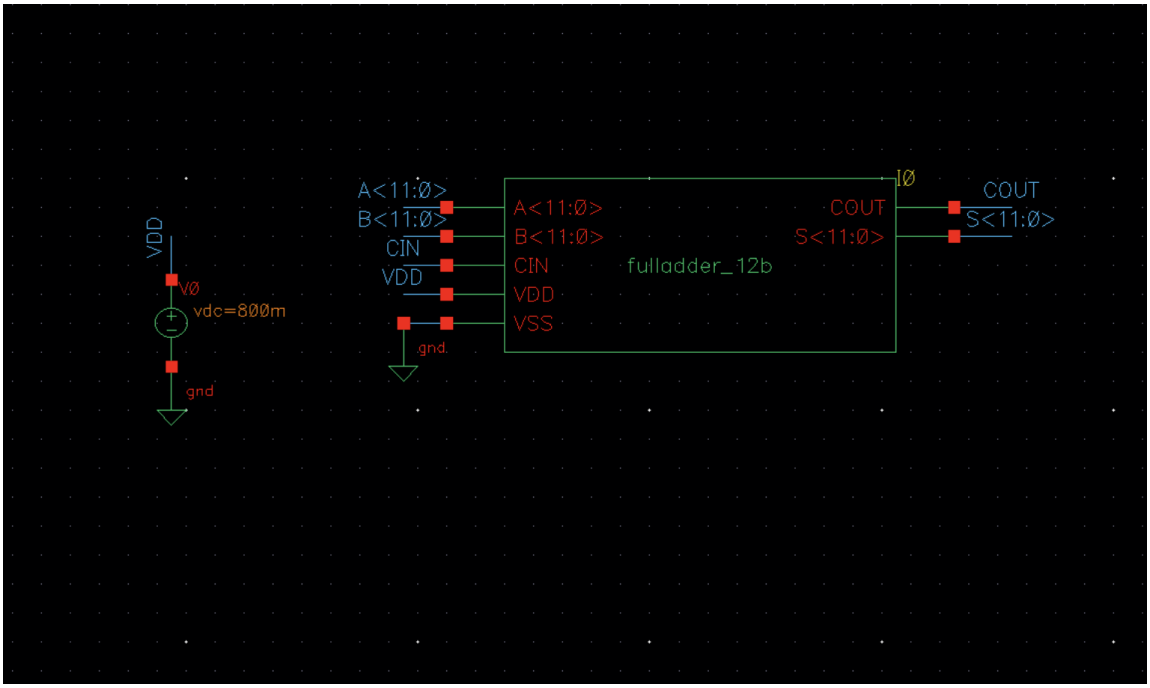


Figure 16: `tb_fulladder_12b`: `fulladder_12b` at $V_{DD} = 0.8$ V. Ports A⟨11:0⟩, B⟨11:0⟩, CIN in; SUM and COUT out.

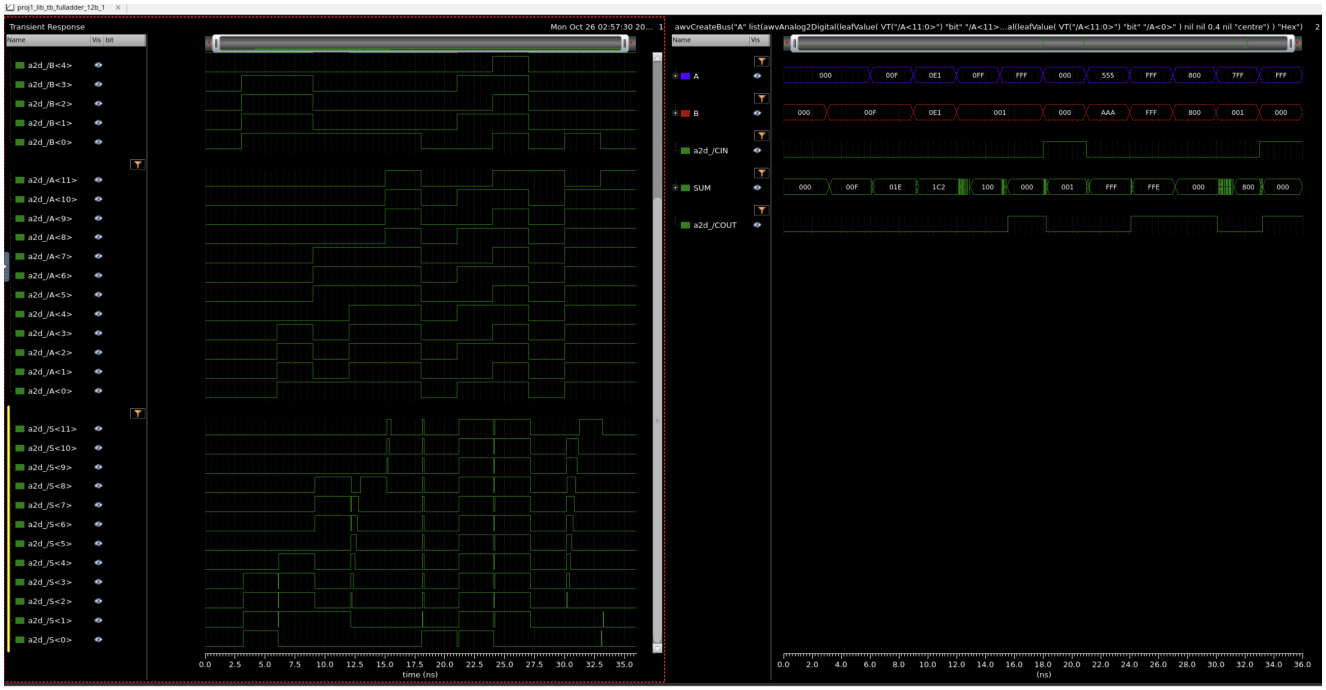


Figure 17: `tb_fulladder_12b`, 0–35 ns: A, B, CIN and SUM as bits (left) and hex buses (right).

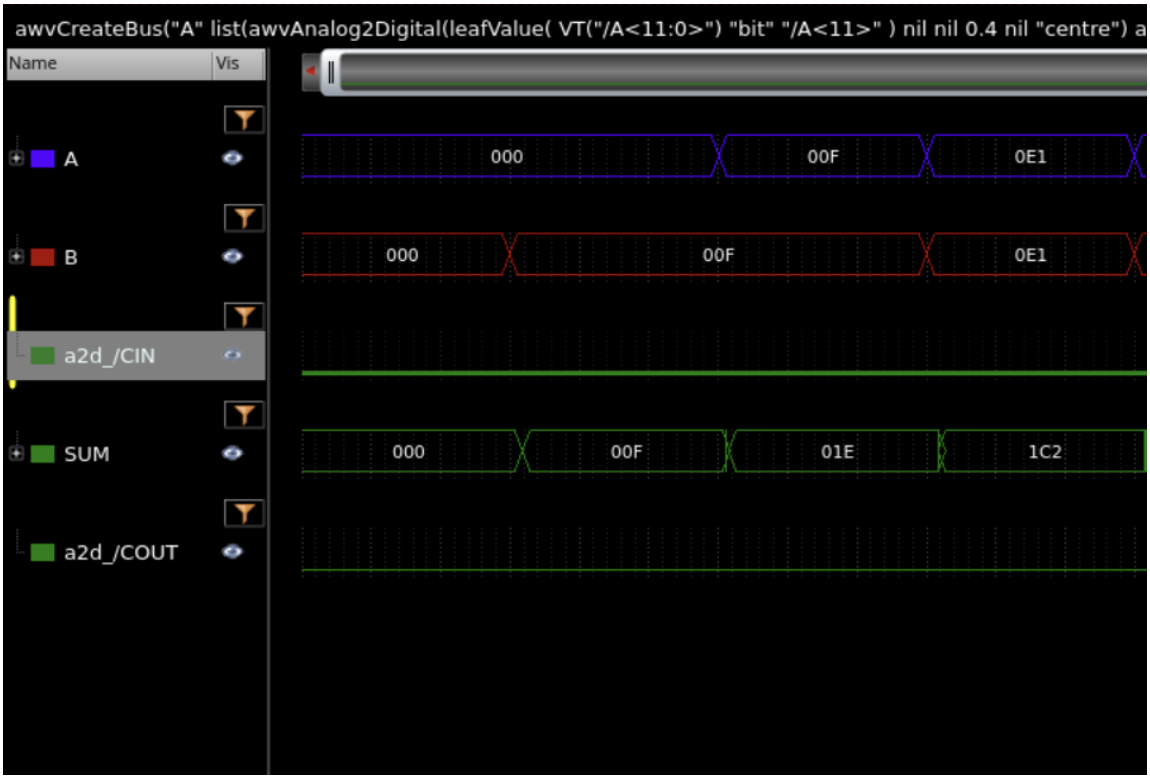


Figure 18: Close-up: 0x00F + 0x00F = 0x01E; 0x0E1 + 0x0E1 = 0x1C2. CIN = 0, COUT = 0.

Logical-effort analysis. Delay of each stage in units of $\tau = 10.5$ ps (Part 2: FO4 = 53 ps = 5τ, chain of `inv_u`) as $d = f + p$, with the effort $f = g\,h$ computed directly from fin counts: $f = R \cdot C_{\text{load}}/C_{\text{inv_u}}$, where R is the gate's worst-edge pull-up or pull-down resistance relative to `inv_u` (PMOS 4 / NMOS 2 fins, $C_{\text{inv_u}} = 6$ fins) and C_{load} the gate fins it drives. A full-adder input or carry-in (one `xor2` input: `inv_2x` 12 fins + two 2-fin devices; plus one `nand2_u` input, 4 fins) is 20 fins.

Stage	Load	R (worst edge)	f	p	d [τ]
`inv_2x` (xor2 input inverter)	4 fins	0.5	0.3	1	1.3
`xor2` core (2-series stacks: PMOS 2+2, NMOS 2+4 fins)	20 fins	2.75 (avg. up 4, down 1.5)	9.2	3	12.2
`nand2_u` (2+2 series, 2‖2)	12 fins	2	4	2	6
`inv_2x` (and2 output)	4 fins	0.5	0.3	1	1.3
`nor2` in `or2` (PMOS 2+2 series)	12 fins	2.5	5	2	7
`inv_2x` (or2 output = C_{out})	20 fins	0.5	1.7	1	2.7

Full-adder arcs: $C_{in} \to C_{out}$ (and2 + or2) $= 6 + 1.3 + 7 + 2.7 \approx 17\tau$; $C_{in} \to S$ (one xor2) $\approx 13\tau$; $A/B \to S$ (two xor2) $\approx 26\tau$; $A/B \to C_{out}$ (xor2, and2, or2) $\approx 30\tau$; a partial-product `and2` into a full-adder input $\approx 6 + 2.7 \approx 9\tau$.

Multiplier critical path. With ripple rows, the longest path alternates carry and sum through all three rows and then ripples along the last one: $a_i b_j$ (and2) $\to$ I5 bit 0 ($A/B \to C_{out}$) $\to$ I5 bit 1 ($C_{in} \to S$) $\to$ I7 bit 0 ($A/B \to C_{out}$) $\to$ I7 bit 1 ($C_{in} \to S$) $\to$ I9 bit 0 ($A/B \to C_{out}$) $\to$ I9 bits 1–3 ($C_{in} \to C_{out}$) $\to P_7$:

$$D_{\text{mult}} \approx 9 + 30 + 13 + 30 + 13 + 30 + 3 \times 17 = 176\,\tau \approx 1.85 \text{ ns}.$$

Measured worst case: 1.14–1.24 ns. The estimate is about 50% high, as expected: it charges every stage its slower edge and the full 20-fin load, and the slowest simulated vector (244) need not excite the whole alternating path. Both agree on what dominates: the `xor2` core, whose 2-fin series PMOS stack makes it about 4× weaker than `inv_u` on the rising edge (12 of the 26 τ in a sum arc). **Sizing from this analysis:** widening the `xor2` stacks to 4-fin PMOS and NMOS ($R \approx 1$) cuts each xor2 stage from $\approx 12\tau$ to $\approx 5\tau$ and the multiplier path to $\approx 120\tau \approx 1.3$ ns (LE); the cells were simulated at the drawn sizes.

Accumulator adder. The product bits arrive at different times; the late ones are the upper bits. Worst case: P_4 settles last, then $A/B \to C_{out}$ at bit 4, carries through bits 5–10 and $C_{in} \to S$ at bit 11: $30 + 6 \times 17 + 13 = 145\,\tau \approx 1.5$

ns. (A carry from ACC bit 0 through all 12 bits, $30 + 11 \times 17 \approx 217\,\tau \approx 2.3$ ns, starts at t_{cq} after the edge and so ends earlier than the path through the multiplier.) The register input adds the `mux2` and `and2` before the flip-flop, $\approx 10\tau \approx 0.1$ ns.

Unit tests and predicted vs. measured.

Block	Bench	Test
Multiplier	`tb_multiplier`	Exhaustive: all 256 input pairs, product compared to $a \cdot b$ (error count must be 0); 0 errors f
12-bit accumulator	`tb_accumulator_12b`	accumulate, RST, hold; $16 \times$ 0xE1 $\rightarrow$ 0xE10

Multiplier bench (`tb_multiplier4b`, Fig. below). Eight `vpulse` sources count through all 256 (A, B) pairs, one per t_v (A the low nibble: bit k has delay $2^k t_v$, width $2^k t_v$, period $2^{k+1} t_v$; B bit k uses $2^{k+4} t_v$), 20 ps edges, tran stop $256\,t_v$; each product bit drives one `inv_2x` (`LD<7:0>`). **[TODO: Inputs are ideal sources; add two `inv_u` buffers per input or state this.]**

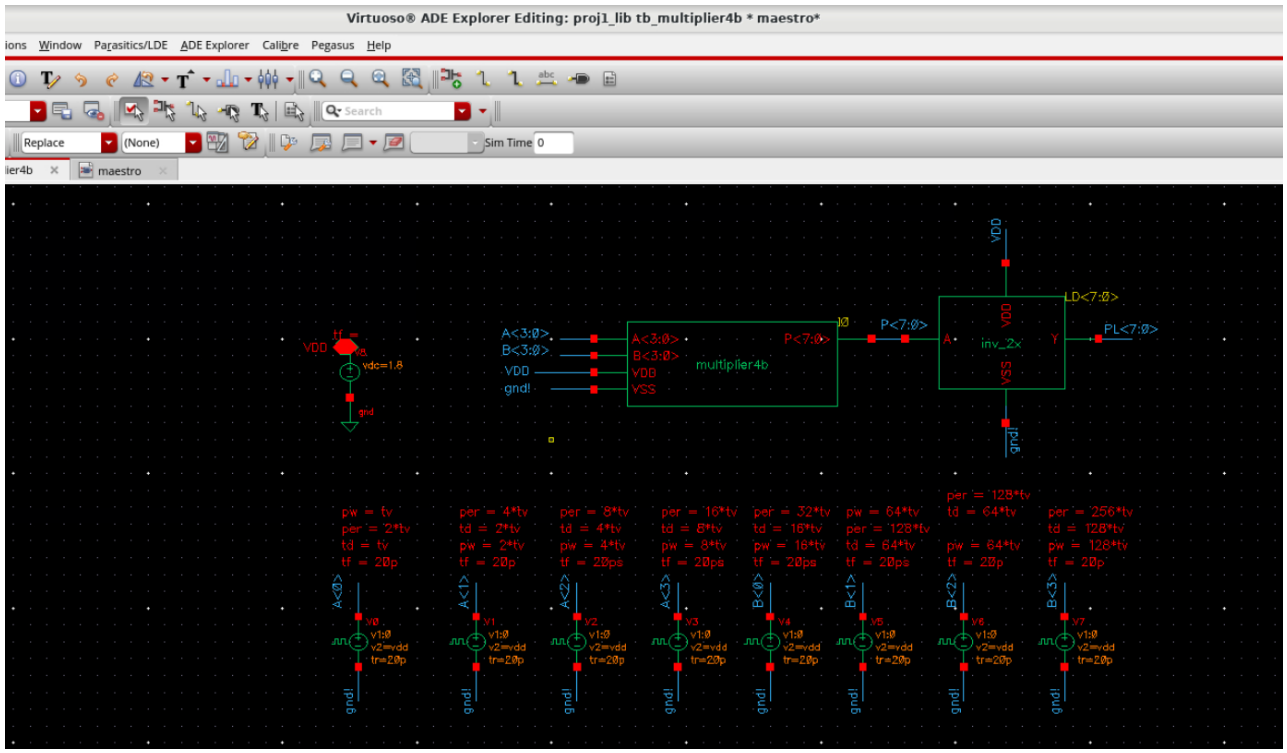


Figure 19: `tb_multiplier4b`. **[TODO: Re-capture with V8 = `vdd` = 0.8 V (this capture shows vdc = 1.8) and V5 corrected to delay/width $32\,t_v$, period $64\,t_v$.]**

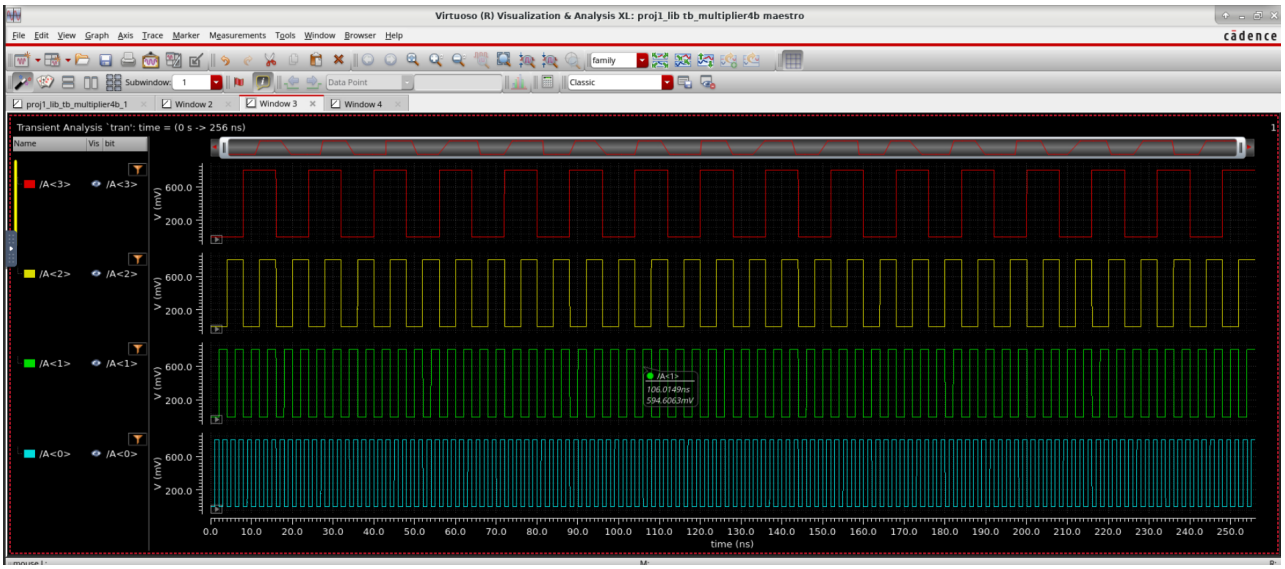


Figure 20: Stimulus $A\langle 3{:}0\rangle$ at $t_v = 1$ ns: A_0 toggles every vector, A_3 every 8.

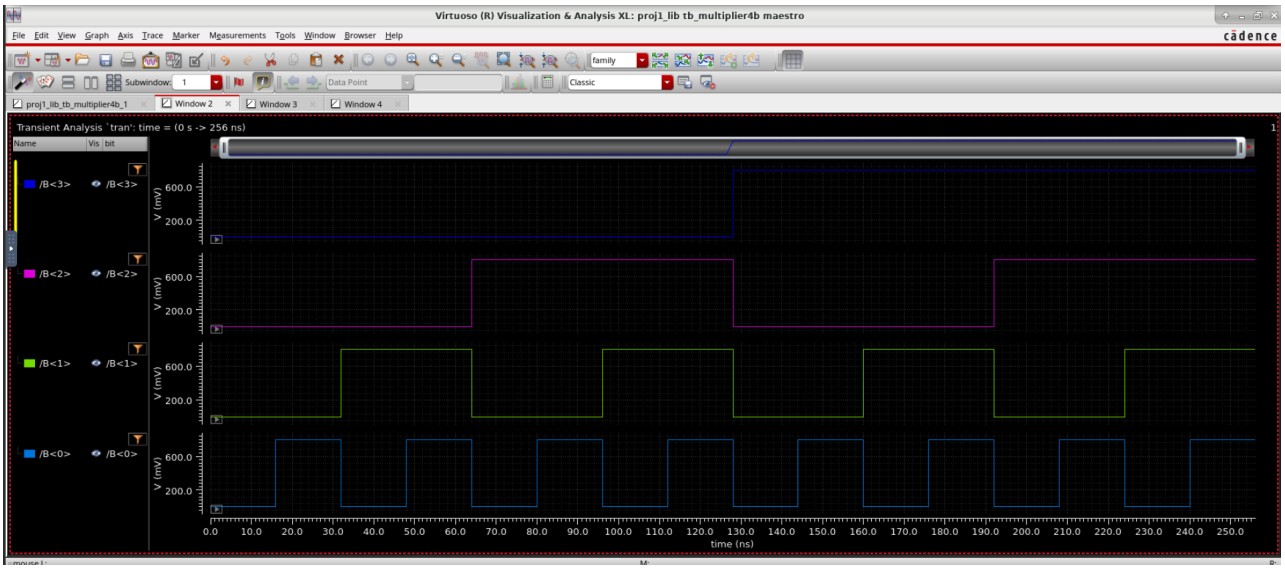


Figure 21: Stimulus $B\langle 3{:}0\rangle$: B steps once every 16 vectors, so all 256 pairs occur in 256 ns.

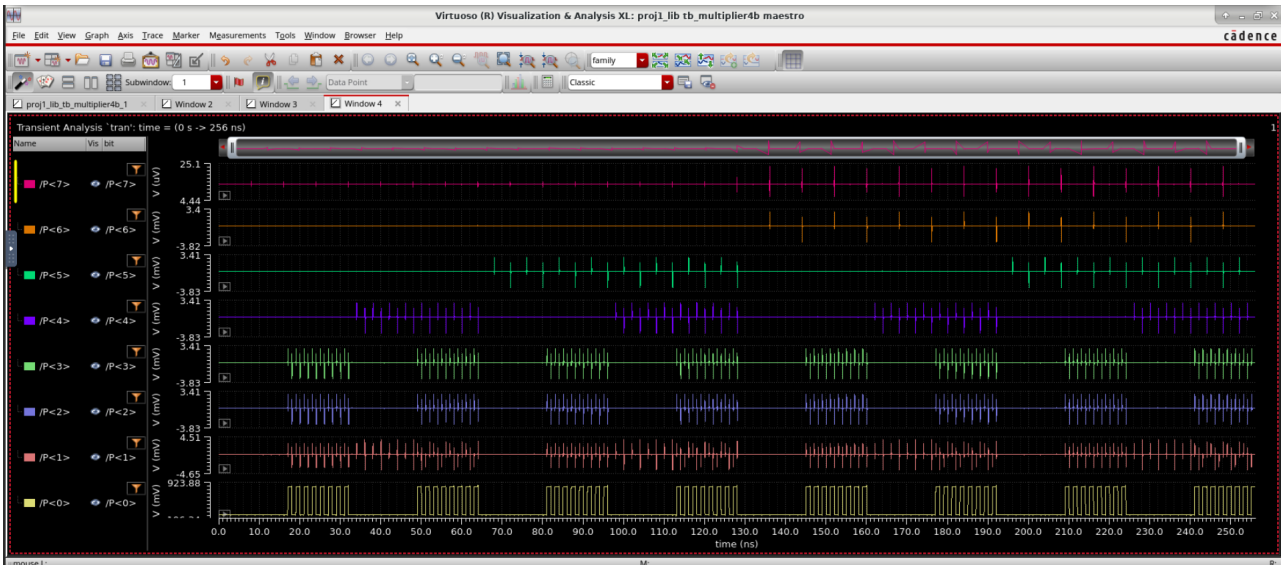


Figure 22: First run (before the fixes): only $P_0 = a_0 b_0$ switches; $P_{1..7}$ stay near 0 V (mV-scale glitches). This located two design errors, below.

Errors found by the first run. (1) `xor2`: the pull-down had the $A/\bar{A}$ devices in parallel, in series with the $B/\bar{B}$ pair in parallel, so it always conducted and fought the pull-up when $A \neq B$; corrected to two series stacks $A \cdot B$ and $\bar{A} \cdot \bar{B}$. (2) `multiplier4b`: all four `and2x4` received $(A\langle 0..3\rangle, B\langle 0..3\rangle)$ and formed $a_i b_i$; each row j now receives B_j on all four B inputs. A logic-level check of the corrected netlist gives 0 wrong products out of 256. **[TODO: Rerun: error count over 256 vectors, worst-case settling time from the t_v sweep, delay of the slowest pair; replace the figure above with the passing run.]**

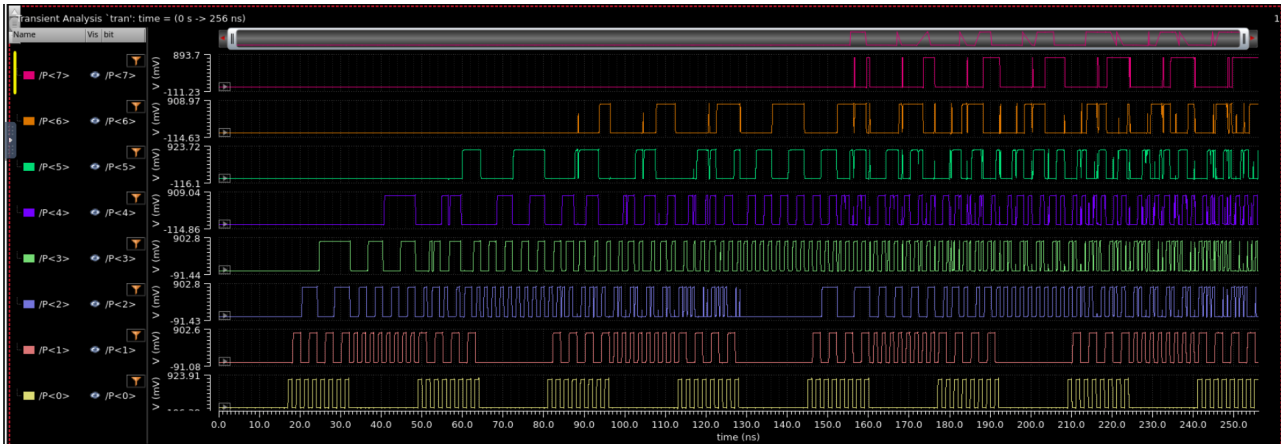


Figure 23: After the fixes ($t_v = 1$ ns): every product bit switches. First rising edges match the hand values: P_1 at vector 18 (2×1), P_3 at 24 (8×1), P_4 at 40 (8×2), P_5 at 59 (11×3), P_7 at 159 ($15 \times 9 = 135$); short spikes are ripple glitches inside a vector. **[TODO: Zoom on the worst-case transition.]**

Exhaustive check. Three ADE expressions turn the bits into integers: $P_{int} = \sum_k 2^k V(P_k)/V_{DD}$, $AB_{int} = (\sum_k 2^k V(A_k)/V_{DD})(\sum_k 2^k V(B_k)/V_{DD})$, and err $= |P_{int} - AB_{int}|$. `errS` samples err once per vector at $0.95\,t_v$ (`sample(err, 0.95 tv, 255.95 tv, "linear", tv)`) and `errmax` = `ymax(errS)` with spec < 0.5.

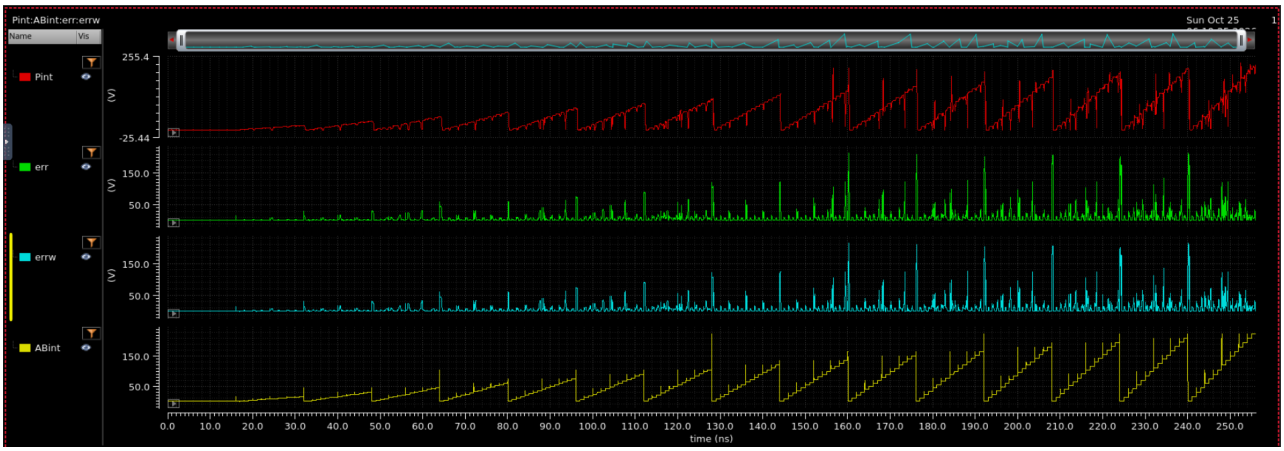


Figure 24: P_{int} follows the AB_{int} staircase in every block of 16 vectors; the err spikes sit at the vector boundaries, where AB_{int} itself spikes because the input bits do not cross at the same instant.

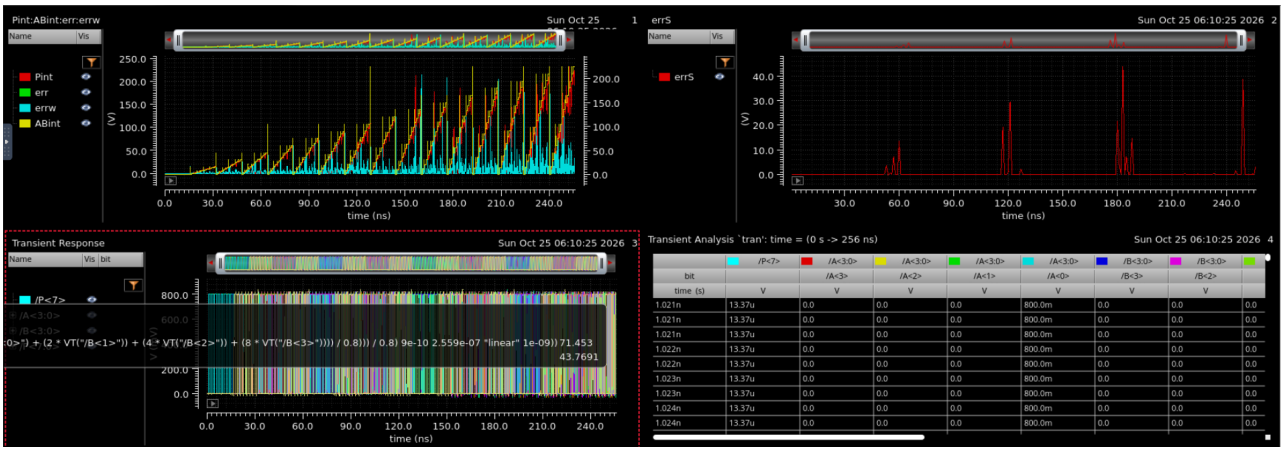


Figure 25: Sampled check at $t_v = 1$ ns (top right, `errS`): 0 for almost all 256 vectors; about ten vectors in four clusters (56–60, 118–122, 180–185, 242–246 ns) are non-zero, peak `errmax` = 43.77.

At $t_v = 1$ ns `errmax` = 43.77 (71.45 with the earlier 0.9 t_v sample point). The non-zero values are fractional, not the power-of-two error a settled wrong bit would give, so those vectors are still switching about 0.95 ns after the inputs: a settling limit, not a logic error.

t_v **sweep.** Tran stop 256 t_v, so every point simulates all 256 vectors; `t_worst` gives the worst vector, $i = \text{round}(t/t_v - 0.95)$.

t_v	sample point ($0.95\,t_v$)	`errmax`	`t_worst`	result (< 0.5)
1.0 ns	0.950 ns	43.77	183.0 ns	fail (vector 182)
1.1 ns	1.045 ns	13.66	269.4 ns	fail (vector 244)
1.2 ns	1.140 ns	0.621	293.9 ns	fail (vector 244)
1.3 ns	**1.235 ns**	**0.170**	323.6 ns	**pass**
1.4 ns	1.330 ns	0.136	348.5 ns	pass
1.5 ns	1.425 ns	0.118	373.4 ns	pass

All 256 products are correct once $t_v \geq 1.3$ ns. The slowest vector is 244 ($B = 15$, A: 0011 → 0100, product 45 → 60): with $B = 15$ all four partial-product rows are active and three A bits switch, so the carry ripples through the whole array into P_4. It is still settling at 1.14 ns and settled by 1.235 ns, so the **measured worst-case multiplier delay is 1.14–1.24 ns (≈ 1.2 ns)**. A second data point: vector 182 (A 5 → 6, $B = 11$) has P_6 crossing $V_{DD}/2$ 962 ps after the input edge (`d_mult` at $t_v = 1$ ns). Typical vectors settle in 0.3–0.6 ns (zoom below). The residual 0.12–0.17 at passing t_v is the bit voltages sitting a few mV off the rails, weighted up to 128.



78 rows

Point	Test	Output	Nominal	Spec	Weight	Pass/Fail
Parameters: tv=1n						
1	proj1_lib_tb_mul...	/P<7>				
1	proj1_lib_tb_mul...	/A<3:0>				
1	proj1_lib_tb_mul...	/B<3:0>				
1	proj1_lib_tb_mul...	/P<7:0>				
1	proj1_lib_tb_mul...	errmax_old	71.45			
1	proj1_lib_tb_mul...	Pint				
1	proj1_lib_tb_mul...	ABint				
1	proj1_lib_tb_mul...	err				
1	proj1_lib_tb_mul...	errw				
1	proj1_lib_tb_mul...	errS				
1	proj1_lib_tb_mul...	errmax	43.77	< 0.5		fail
1	proj1_lib_tb_mul...	tworst	183n	< (256 * tv)		error
1	proj1_lib_tb_mul...	d_mult	962.1p			
Parameters: tv=1.1n						
2	proj1_lib_tb_mul...	/P<7>				
2	proj1_lib_tb_mul...	/A<3:0>				
2	proj1_lib_tb_mul...	/B<3:0>				
2	proj1_lib_tb_mul...	/P<7:0>				
2	proj1_lib_tb_mul...	errmax_old	206			
2	proj1_lib_tb_mul...	Pint				
2	proj1_lib_tb_mul...	ABint				
2	proj1_lib_tb_mul...	err				
2	proj1_lib_tb_mul...	errw				
2	proj1_lib_tb_mul...	errS				
2	proj1_lib_tb_mul...	errmax	13.66	< 0.5		fail
2	proj1_lib_tb_mul...	tworst	269.4n	< (256 * tv)		error
2	proj1_lib_tb_mul...	d_mult	eval err			
Parameters: tv=1.2n						
3	proj1_lib_tb_mul...	/P<7>				
3	proj1_lib_tb_mul...	/A<3:0>				
3	proj1_lib_tb_mul...	/B<3:0>				
3	proj1_lib_tb_mul...	/P<7:0>				
3	proj1_lib_tb_mul...	errmax_old	158.9			
3	proj1_lib_tb_mul...	Pint				
3	proj1_lib_tb_mul...	ABint				
3	proj1_lib_tb_mul...	err				
3	proj1_lib_tb_mul...	errw				
3	proj1_lib_tb_mul...	errS				
3	proj1_lib_tb_mul...	errmax	621m	< 0.5		fail
3	proj1_lib_tb_mul...	tworst	293.9n	< (256 * tv)		error
3	proj1_lib_tb_mul...	d_mult	669.2p			
Parameters: tv=1.3n						
4	proj1_lib_tb_mul...	/P<7>				
4	proj1_lib_tb_mul...	/A<3:0>				
4	proj1_lib_tb_mul...	/B<3:0>				
4	proj1_lib_tb_mul...	/P<7:0>				
4	proj1_lib_tb_mul...	errmax_old	164.2			
4	proj1_lib_tb_mul...	Pint				
4	proj1_lib_tb_mul...	ABint				
4	proj1_lib_tb_mul...	err				
4	proj1_lib_tb_mul...	errw				
4	proj1_lib_tb_mul...	errS				
4	proj1_lib_tb_mul...	errmax	170.3m	< 0.5		pass
4	proj1_lib_tb_mul...	tworst	323.6n	< (256 * tv)		error
4	proj1_lib_tb_mul...	d_mult	eval err			
Parameters: tv=1.4n						
5	proj1_lib_tb_mul...	/P<7>				
5	proj1_lib_tb_mul...	/A<3:0>				
5	proj1_lib_tb_mul...	/B<3:0>				
5	proj1_lib_tb_mul...	/P<7:0>				
5	proj1_lib_tb_mul...	errmax_old	89.95			
5	proj1_lib_tb_mul...	Pint				
5	proj1_lib_tb_mul...	ABint				
5	proj1_lib_tb_mul...	err				
5	proj1_lib_tb_mul...	errw				
5	proj1_lib_tb_mul...	errS				
5	proj1_lib_tb_mul...	errmax	136.2m	< 0.5		pass
5	proj1_lib_tb_mul...	tworst	348.5n	< (256 * tv)		error
5	proj1_lib_tb_mul...	d_mult	eval err			
Parameters: tv=1.5n						
6	proj1_lib_tb_mul...	/P<7>				
6	proj1_lib_tb_mul...	/A<3:0>				
6	proj1_lib_tb_mul...	/B<3:0>				
6	proj1_lib_tb_mul...	/P<7:0>				
6	proj1_lib_tb_mul...	errmax_old	63.12			
6	proj1_lib_tb_mul...	Pint				
6	proj1_lib_tb_mul...	ABint				
6	proj1_lib_tb_mul...	err				
6	proj1_lib_tb_mul...	errw				
6	proj1_lib_tb_mul...	errS				
6	proj1_lib_tb_mul...	errmax	118.4m	< 0.5		pass
6	proj1_lib_tb_mul...	tworst	373.4n	< (256 * tv)		error
6	proj1_lib_tb_mul...	d_mult	eval err			

Figure 26: ADE Results for the sweep: `errmax` (spec < 0.5) fails at 1.0–1.2 ns and passes from 1.3 ns. (The "error" on `t_worst` is only its spec, which cannot use a design variable; `d_mult` uses a fixed 182–183 ns window and is valid only at $t_v = 1$ ns; `errmax_old` uses fixed sample times.)

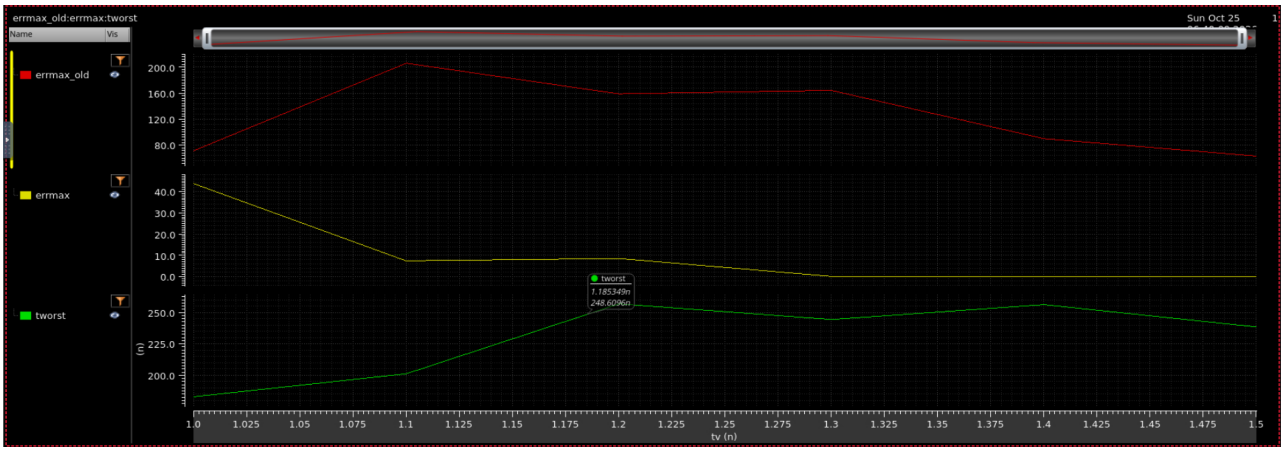


Figure 27: `errmax` and `t_worst` against t_v from an earlier run with a fixed 256 ns stop (covers fewer vectors above 1 ns); kept for the shape of the curve, the table above is the result.

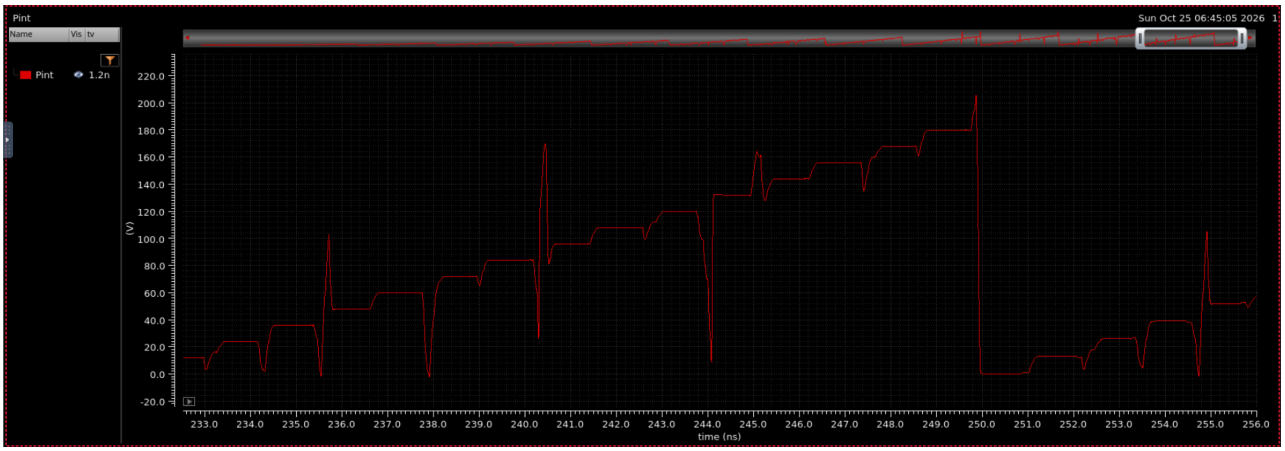


Figure 28: P_{int} at $t_v = 1.2$ ns, 233–256 ns ($B = 12$, then $B = 13$): each product settles 0.3–0.6 ns after its vector edge; the spike near 249.9 ns is the 180 → 0 step where all four A bits and B_0 flip.

[TODO: Zoom on vector 244 at $t_v = 1.3$ ns (317.2–318.5 ns): which P bit settles last, input-edge-to-0.4 V delay (the measured critical path for the logical-effort comparison).] **12-bit adder vectors.** Hand checks on the bus plot: 0x00F + 0x00F = 0x01E; 0x0E1 + 0x0E1 = 0x1C2; 0x0FF + 0x001 = 0x100; 0x555 + 0xAAA = 0xFFF; 0xFFF + 0xFFF sets SUM =0xFFE and COUT; 0x7FF + 0x001 = 0x800. The ripple adder is logically correct.

Accumulator (`tb_accumulator_12b`). DUT is `accumulator_12b`: CLK, EN, PROD⟨7:0⟩, RST in; ACC⟨11:0⟩ out; $V_{DD} = 0.8$ V.

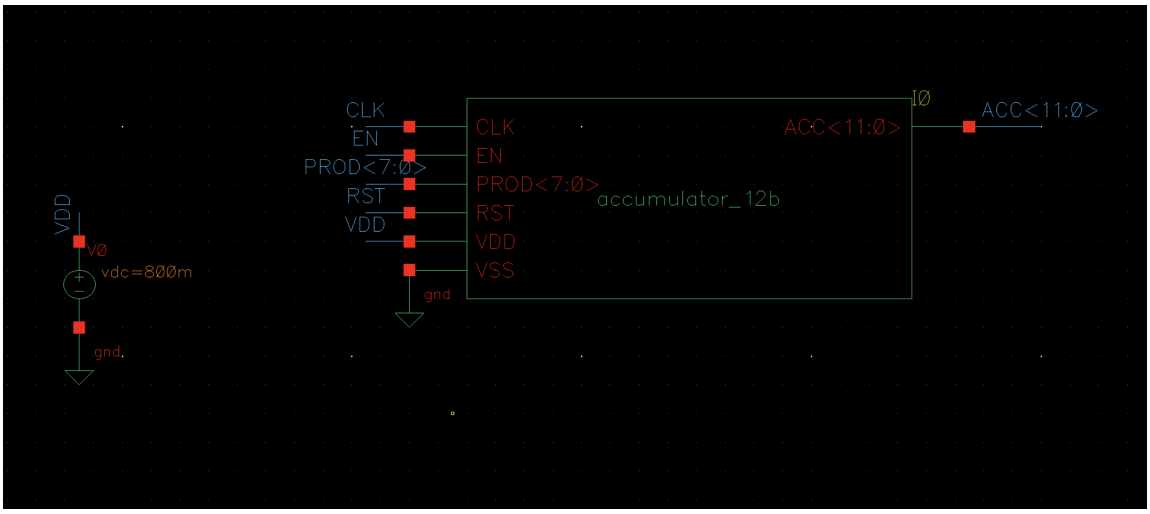


Figure 29: `tb_accumulator_12b`: `accumulator_12b` with product byte PROD⟨7:0⟩ and 12-bit ACC.

After RST, PROD = 0x0F (15) is added while EN is high: ACC = 0x00F, 0x01E, 0x02D, 0x03C. EN falls (hold), RST clears to 0. Then PROD = 0xE1 ($15 \times 15 = 225$) for sixteen clocks: ACC steps 0x0E1, 0x1C2, …, 0xD2F, 0xE10. $16 \times 225 = 3600 = 0xE10$, the Test B full-scale sum, with carry into the upper nibble.



Figure 30: `tb_accumulator_12b`, 0–100 ns: ACC and PROD bits (left); CLK, EN, RST and hex ACC/PROD (right).

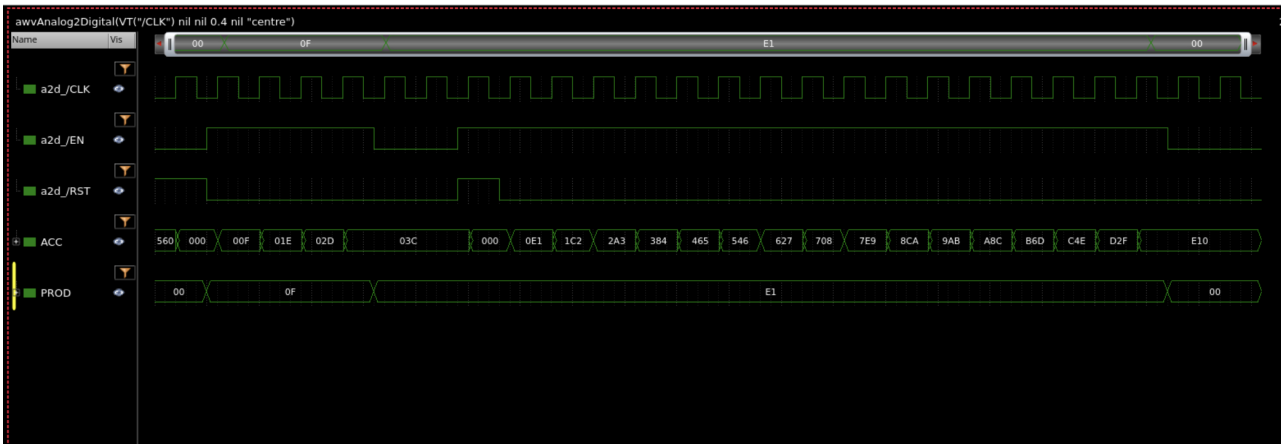
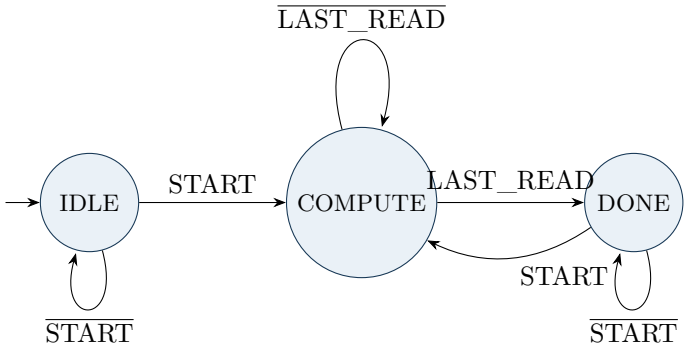


Figure 31: Hex close-up: four adds of 0x0F, RST, then sixteen adds of 0xE1 to 0xE10.

4. Sequencer (3.3)

Rubric (3.3): 4-bit address counter and control FSM (idle, compute, done) built from the registers; state diagram; timing of START, counter, memory reads, accumulator update and DONE over one dot product.

State diagram.



Encoding (from `controller_mac`): two state registers, S_0 and DONE; IDLE = $(S_0, \text{DONE}) = (0, 0)$, COMPUTE: RUN = $S_0\overline{\text{DONE}} = 1$, DONE = $(0, 1)$. Two more registers carry the pipeline: VALID (= RUN one cycle late, so the first product is accumulated one cycle after its read) and LAST_READ (set the cycle after the counter reaches 15). RST clears S_0 and DONE from any state. Per state:

- **IDLE / DONE, START high:** $G_0 = \text{START} \cdot \overline{\text{RUN}}$ raises CLEAR (clears the accumulator, the counter, VALID and LAST_READ) and sets S_0; DONE clears. START is ignored while RUN.
- **COMPUTE:** COUNT_EN = RUN $\cdot \overline{\text{LAST}}$ steps the counter 0 to 15 and holds it at 15 (LAST = count = 15); ACC_EN = RUN $\cdot$ VALID; S_0 stays set while LAST_READ = 0.
- **Leaving COMPUTE:** FINISH = RUN $\cdot$ LAST_READ sets DONE and clears S_0, so the exit is one cycle after the count reaches 15, which leaves time to accumulate the last product.
- **DONE:** held by DONE $\cdot \overline{\text{START}}$; ACC holds because ACC_EN = 0.

Counter and controller. `counter_4b` is a synchronous toggle-chain counter: $T_1 = EN \cdot Q_0$, $T_2 = T_1 Q_1$, $T_3 = T_2 Q_2$ (`and2`), $D_k = (Q_k \oplus T_k) \cdot \overline{RST}$ with $T_0 = EN$ (`xor2`, `and2`), four `d_ff`; RST is driven by CLEAR, so the count restarts at 0 on START (728 fins). `controller_mac` is the logic above: 4 `register_1b`, 11 `and2`, 3 `or2`, 5 `inv_2x` (948 fins). In `top`, four `mux2` send COUNT to the memory address while RUN = 1 and the external ADDR otherwise, and LOAD_EN = $\overline{\text{RUN} + \text{DONE} + \text{START}}$ gates WE_W and WE_X. The counter passes all 38 vector checks (below). The controller, closed-loop with the counter, follows the expected sequence edge for edge, with DONE 17 edges after START (below).



Figure 32: `counter_4b`: four `d_ff` (I5–I8); $D_k = (Q_k \oplus T_k) \cdot \overline{RST}$ through `xor2` and `and2` (I12/I13, I21/I22, I25/I26, I24/I23); toggle chain $T_1 = EN \cdot Q_0$, $T_2 = T_1 Q_1$, $T_3 = T_2 Q_2$ (I15, I27, I28); RST inverted by I20. 728 fins.

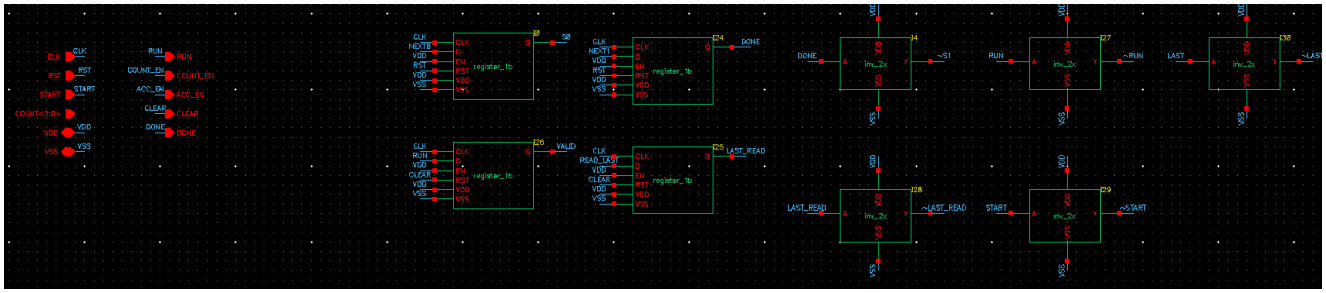


Figure 33: `controller_mac`, state registers: four `register_1b` (S_0 from NEXT0, DONE from NEXT1, both reset by RST; VALID from RUN and LAST_READ from READ_LAST, both reset by CLEAR; EN tied to VDD) and the inverters $\overline{S_1}$ = $\overline{\text{DONE}}$, $\overline{\text{RUN}}$, $\overline{\text{LAST}}$, $\overline{\text{LAST_READ}}$, $\overline{\text{START}}$.

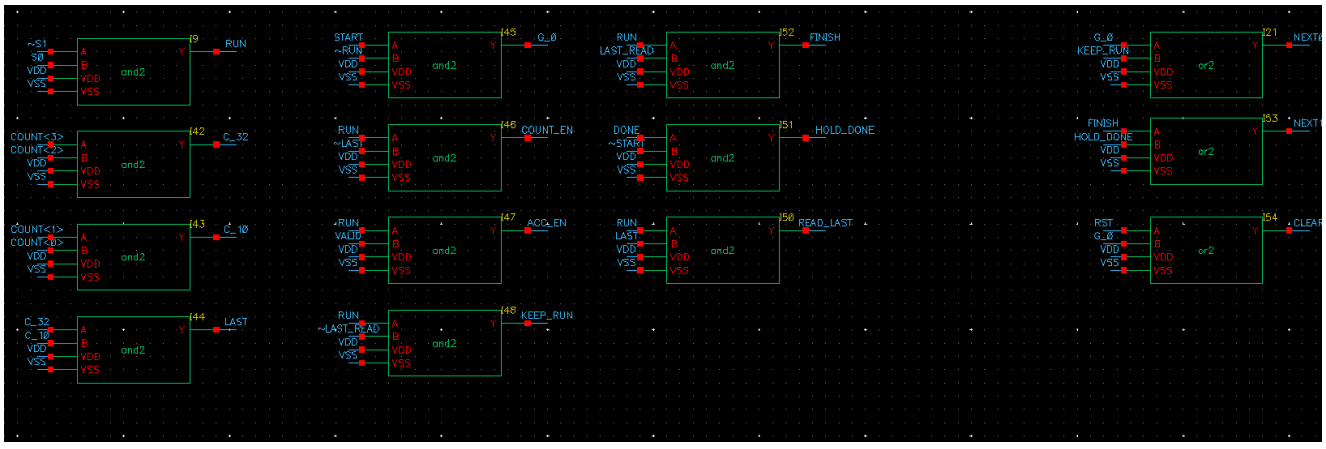


Figure 34: `controller_mac`, next-state and output logic: RUN = $\overline{S_1} S_0$; LAST = $C_{32} C_{10}$ (count = 15); G_0 = START · $\overline{\text{RUN}}$; COUNT_EN = RUN · $\overline{\text{LAST}}$; ACC_EN = RUN · VALID; KEEP_RUN, FINISH, HOLD_DONE, READ_LAST; NEXT0 = G_0 + KEEP_RUN, NEXT1 = FINISH + HOLD_DONE, CLEAR = RST + G_0 (`and2`, `or2`).

`tb_counter4b`. Stimulus and check from a Spectre digital vector file (`counter4b.vec`): CLK (1 ns period, rising edges at 0.5, 1.5, … ns), RST and EN change while CLK is low, and COUNT⟨3:0⟩ is compared with the expected value a quarter period after every edge (output thresholds 0.2/0.6 V). The 38-edge sequence covers reset, counting

through the 15 → 0 wrap, hold (EN = 0), resume, reset while counting and a final hold; expected COUNT: 0 0 1 2 … F 0 1 2 3 4 | 4 4 4 | 5 … 9 | 0 0 | 1 2 3 4 | 4 4. **Result: 38/38 checks pass** (empty `.vecerr`, Spectre 0 errors, 0 warnings); outputs swing rail to rail with $V_{DD} = 0.8$ V.



Figure 35: `tb_counter4b`, 0–39 ns: COUNT⟨3:0⟩. Bit 0 toggles every edge, bit 3 is high for counts 8–15 (9.6–17.6 ns) and 8–9 (28.6–30.6 ns); flat during the EN-low holds (21.6–25.6 ns, after 35.6 ns); reset to 0 at 30.6 ns.



Figure 36: Bit 0 next state D_0 and the master-latch node (`I5/net5`) change before each rising edge and COUNT⟨0⟩ follows after it; V_{DD} flat at 0.8 V.

```
vwanjohi@epyc-1-1:/vast/projects/radway/ese-5700-fall/tmarhguy/project_1$ R=$(ls -td ~/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.* | head -1)
grep -i -n "vec" $(find $R -name "spectre.out") | head -20
find $R -name "*.vecerr" -exec sh -c 'echo "== $1 ($(wc -l < "$1") lines)"; head "$1"' _ {} \;
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/1/proj1_lib_tb_counter_4b_1/psf/spectre.out:106:Reading file:  /vast/home/v/vwanj
ohi/simulation/stimuli/counter4b.vec
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/1/proj1_lib_tb_counter_4b_1/psf/spectre.out:108:Notice from spectre during Digita
l Vector read-in.
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/1/proj1_lib_tb_counter_4b_1/psf/spectre.out:109:    Process Vector Files.
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/1/proj1_lib_tb_counter_4b_1/psf/spectre.out:110:    File read: /vast/home/v/vwanj
ohi/simulation/stimuli/counter4b.vec
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/psf/proj1_lib_tb_counter_4b_1/psf/spectre.out:106:Reading file:  /vast/home/v/vwa
njohi/simulation/stimuli/counter4b.vec
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/psf/proj1_lib_tb_counter_4b_1/psf/spectre.out:108:Notice from spectre during Digi
tal Vector read-in.
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/psf/proj1_lib_tb_counter_4b_1/psf/spectre.out:109:    Process Vector Files.
/vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/psf/proj1_lib_tb_counter_4b_1/psf/spectre.out:110:    File read: /vast/home/v/vwa
njohi/simulation/stimuli/counter4b.vec
== /vast/home/v/vwanjohi/simulation/proj1_lib/tb_counter_4b/maestro/results/maestro/ExplorerRun.0/1/proj1_lib_tb_counter_4b_1/netlist/input_tran.vecerr (0 lines)
vwanjohi@epyc-1-1:/vast/projects/radway/ese-5700-fall/tmarhguy/project_1$
```

Open file in editor (cmd + click)

Figure 37: Vector check: Spectre reads `counter4b.vec`; the mismatch file `input_tran.vecerr` has 0 lines.

Debug note: the first runs left the bench's VSS net unconnected to the simulator ground, so the supply pair floated against the vector-driven inputs (nodes at 0.7–1.5 V, no counting); tying VSS to `gnd` fixed it. **`tb_controller_mac.`** `controller_mac` and `counter_4b` wired as in `top` (COUNT_EN → EN, CLEAR → counter RST), driven and checked by a 60-edge vector file (`controller.vec`, 1 ns clock): reset, START at edge 5, a full dot product, restart from DONE at edge 26, a START at edge 31 during RUN, RST at edge 35, and a second full run from edge 40. RUN, DONE, ACC_EN and COUNT are checked after every edge.

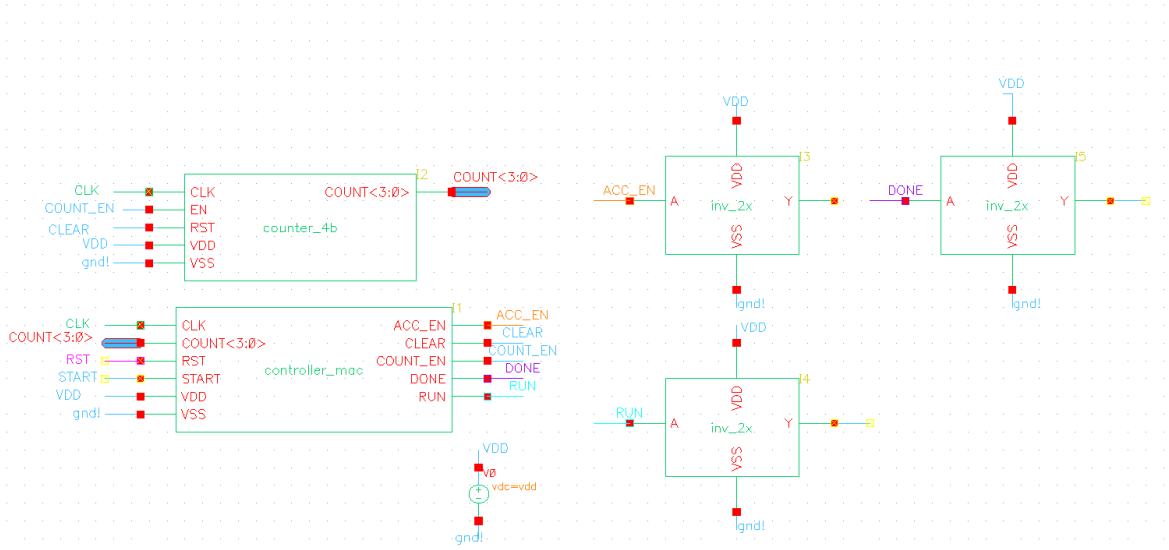


Figure 38: `tb_controller_mac`: `controller_mac` (I1) and `counter_4b` (I2) as in `top` (COUNT_EN → EN, CLEAR → counter RST, COUNT⟨3:0⟩ fed back); `inv_2x` loads on RUN, DONE and ACC_EN; $V_{DD} = 0.8$ V to `gnd!`. CLK, RST and START come from the vector file.

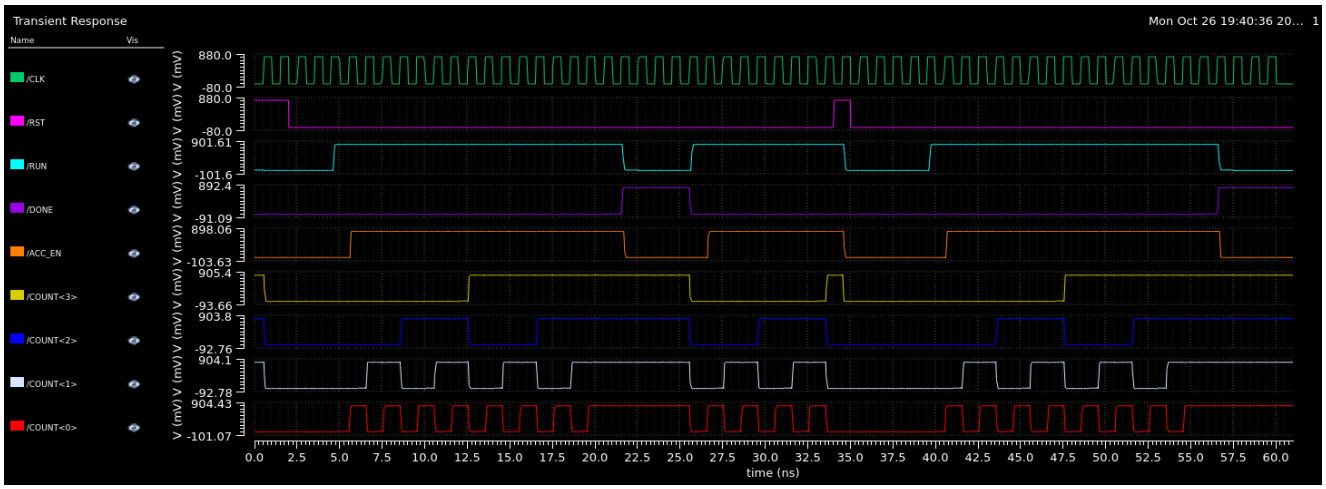


Figure 39: `tb_controller_mac`, 0–61 ns, with COUNT⟨3:0⟩: the counter steps 0→15 and holds during each run, clears on the START edge (25.5 ns) and on RST (34.5 ns). RUN rises after the START edge (4.5 ns), ACC_EN one cycle later (pipeline), and DONE after edge 22 (21.5 ns): **17 edges from START to DONE**. START at edge 26 restarts from DONE; START at edge 31 is ignored while RUN; RST at edge 35 aborts the run; the run from edge 40 ends with DONE at edge 57 and DONE holds.

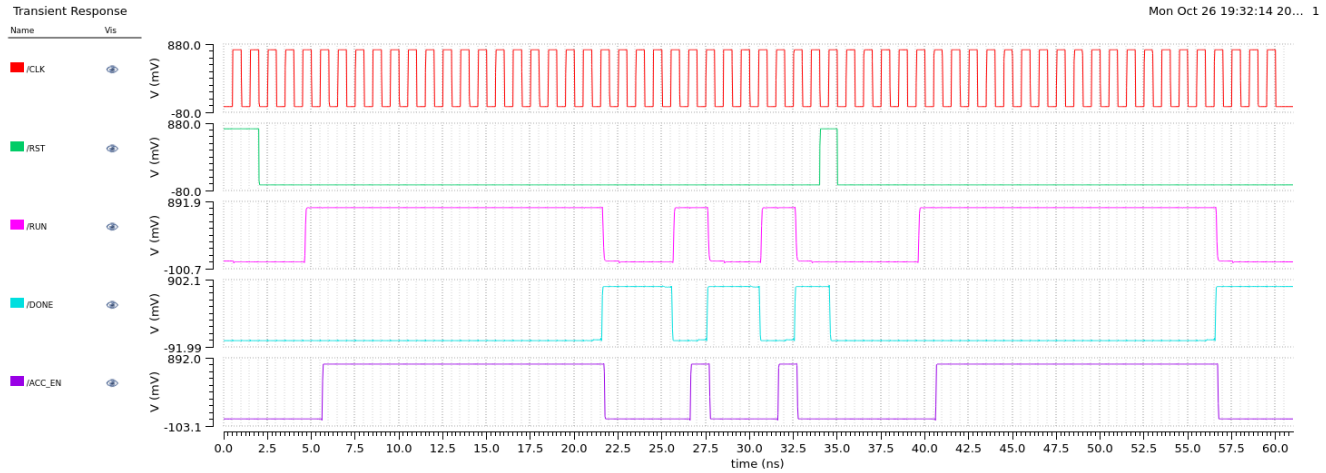


Figure 40: First run, with the counter's reset wired to RST instead of CLEAR: the restart from DONE lasts two cycles because the counter still holds 15 (LAST is already true). Wiring the counter reset to CLEAR, as in `top`, fixed it.

Result: all 60 edges match (`input_tran.vecerr` has 0 lines; Spectre read and processed `controller.vec`).

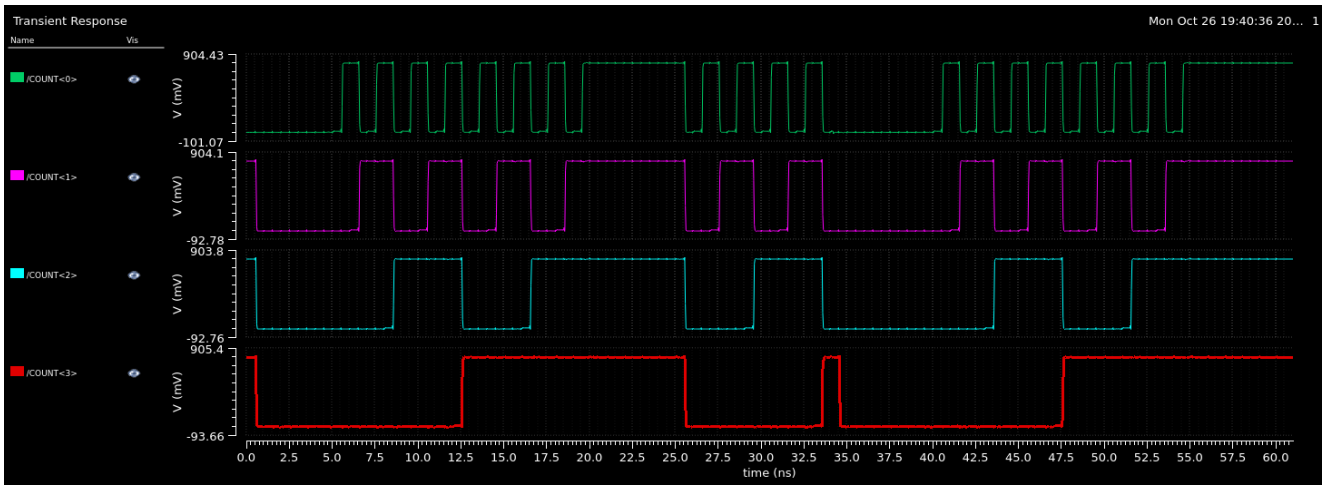


Figure 41: COUNT⟨3:0⟩ in `tb_controller_mac`: 0→15 and hold in each run, cleared at the restart (25.5 ns) and by RST (34.5 ns).

```
vwanjohi@epyc-6-9:/vast/projects/radway/ese-5700-fall/tmarhguy/project_1/High_Quality_Screenshots$ find $R -name "*.vecerr" |
 xargs wc -l
0 /vast/home/v/vwanjohi/simulation/proj1_lib/tb_controller_mac/maestro/results/maestro/ExplorerRun.0/1/proj1_lib_tb_controlle
r_mac_1/netlist/input_tran.vecerr
vwanjohi@epyc-6-9:/vast/projects/radway/ese-5700-fall/tmarhguy/project_1/High_Quality_Screenshots$
```

Figure 42: Vector check for `tb_controller_mac`: the mismatch file has 0 lines.

Timing over one dot product. **[TODO: Annotated diagram, as in Part 2 (`timing_diagram_2p2c.tex` is the model): CLK, START, state, counter, ADDR to the macros, W/X outputs, product, ACC, DONE; mark the START edge, the first read, the first accumulate, the last accumulate, DONE rising.]**

Architecture	pipelined: address and read in cycle n, multiply-accumulate in $n+1$ (VALID)
Cycles START to DONE	17 edges after the START edge (`tb_controller_mac`; limit 20)
Cycles per dot product	17 (plus 32 load cycles when the memories are reloaded)

5. Timing closure (3.4)

Rubric (3.4): critical path identified; minimum period by hand from $t_{CLK\to Q}$/SRAM access, the logical-effort datapath estimate, and register t_{cq} and t_{su}; measured as the shortest passing period; hold checked on fast paths.

Critical path. The pipeline has no register between the macros and the multiplier: the macro's own address latch captures COUNT at the rising edge and its output latch holds DOUT for the rest of the cycle. The paths that must fit in one period are therefore (1) counter t_{cq} → `mux2` → macro address latch (short), and (2) CLK edge → SRAM access → multiplier → 12-bit adder → accumulator setup, the critical path: $T_{\min} \geq t_{\text{SRAM}} + t_{\text{mult}} + t_{\text{add}} + t_{su}$, with t_{SRAM} measured from CLK and so already including the macro's internal address latch.

$$T_{\min} \geq t_{cq}+t_{\text{SRAM}}+t_{\text{mult}}+t_{\text{add}}+t_{su}+t_{\text{skew}} \quad \text{(non-pipelined)}; \qquad T_{\min} \geq \max\left(t_{cq}+t_{\text{SRAM}}+t_{su},\ t_{cq}+t_{\text{mult}}+t_{\text{add}}+t_{su}\right) \quad \text{(pipelined)} \tag{1}$$

Term	Value	Source
t_{cq} (register)	≈ 100 ps	§2, `tb_d`
t_{SRAM} (CLK to DOUT)	488 ps	Part 2,
t_{mult}	1850 ps (LE); 1140–1240 ps measured	§3
t_{add} ($P_4 \to ACC_{11}$) + register input	≈ 1500 + 100 ps (LE)	§3
t_{su}	**[TODO: ps]**	§2
Hand $T_{\min}$	$0.49 + 1.85 + 1.6 + t_{su} \approx$ **4.0** ns $+ t_{su}$ (LE); ≈ 3.3 ns $+ t_{su}$ with the measured multiplier	t_{SRAM} +
Measured $T_{\min}$	**[TODO: ns]**	shortest

Period sweep. **[TODO: Table as in Part 2 §6: t_{clk}, ACC error for A/B/C, result.]** Discuss the gap between hand and measured value: **[TODO:]**.

[missing figure: tb_top_tclk_sweep]

Figure 43: **[TODO: Pass/fail against clock period.]**

Hold check at the fast paths. Two short paths: (1) counter → `mux2` → macro address latch, which closes at the same rising edge that changes the counter, so the new count must arrive after the latch has closed: $t_{cq,\min}+t_{\text{mux}} \geq t_{h,\text{latch}}$; (2) register to register in the controller and counter (e.g. VALID from RUN through one `and2`): $t_{cq,\min}+t_{\text{logic},\min} \geq t_h+t_{\text{skew}}$. **[TODO: Margins in ps from `tb_dff` ($t_{cq,\min}$, t_h) and a simulation of path (1) in `tb_top`.]**

6. Verification (3.5)

Rubric (3.5): three test vectors; ACC and DONE waveforms for Test A; pass/fail table for all three.

Bench (`tb_top`). DUT is `top`. CLK is a `vpulse` (0 → 0.8 V, $t_r = t_f = 20$ ps); $V_{DD} = 0.8$ V. RST, START, WE_W, WE_X, ADDR⟨3:0⟩ and DIN⟨3:0⟩ each come from a `pwlFile`: load (START low, WE_W then WE_X writing 16 words) then compute (START pulse, wait for DONE, sample ACC). ACC⟨11:0⟩ and DONE are observed unloaded in this capture.

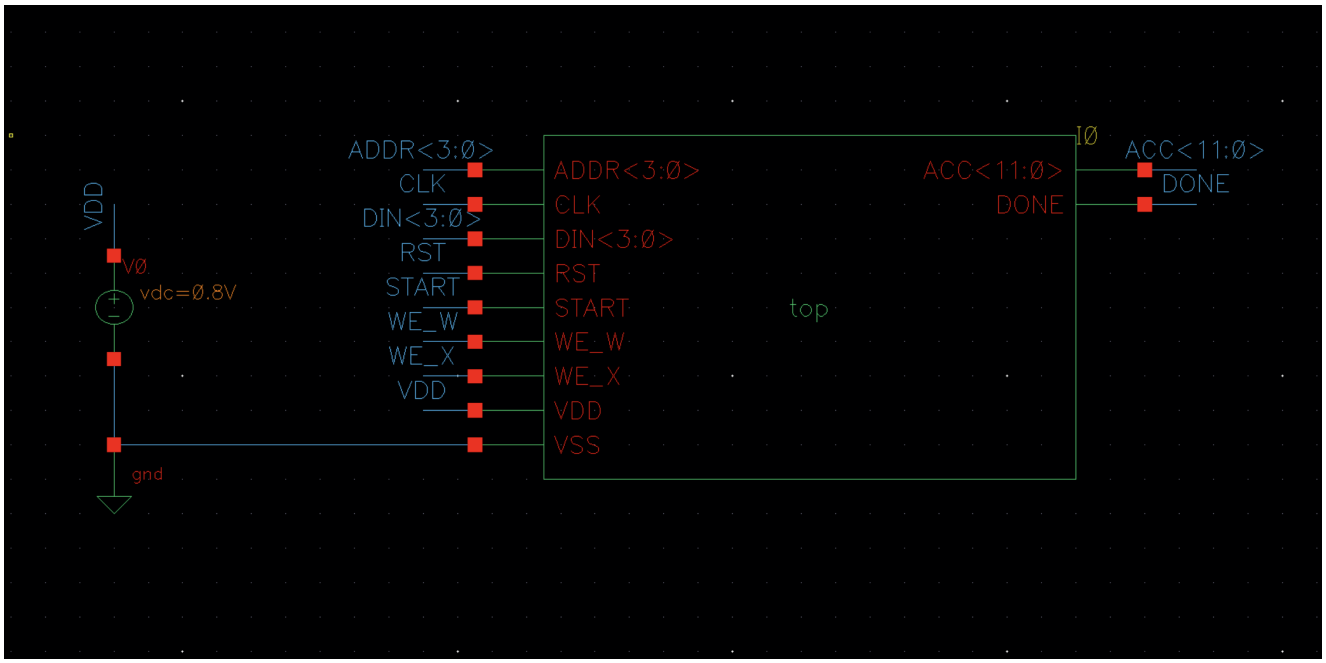


Figure 44: `tb_top`: `top` driven by CLK (V1) and $V_{DD} = 0.8$ V (V0); RST, START, WE_W, WE_X, ADDR⟨3:0⟩ and DIN⟨3:0⟩ from PWL files. Outputs ACC⟨11:0⟩ and DONE.

Test	W[0..15]	X[0..15]	Expected ACC
A	3, 12, 7, 0, 15, 9, 1, 14, 6, 11, 2, 13, 8, 5, 10, 4	9, 2, 15, 7, 4, 12, 0, 11, 13, 1, 8, 6, 3, 14, 5, 10	845 (0x34D)
B	all 15	all 15	3600 (0xE10)
C	5, 10, 5, 10, …	10, 5, 10, 5, …	800 (0x320)

Test	Expected	ACC read	DONE high at cycle	Cycles	Result
A	845 (0x34D)	0x34D	edge 17 after START (502 ns)	17	**pass** ($T = 10$ ns)
B	3600 (0xE10)	0xE10	edge 17 after START (502 ns)	17	**pass** ($T = 10$ ns)
C	800 (0x320)	0x320	edge 17 after START (502 ns)	17	**pass** ($T = 10$ ns)

Test B is the full-scale case (largest carry chains, no overflow); Test C alternates 5×10 and 10×5 to maximize toggling on the memory outputs and multiplier inputs.

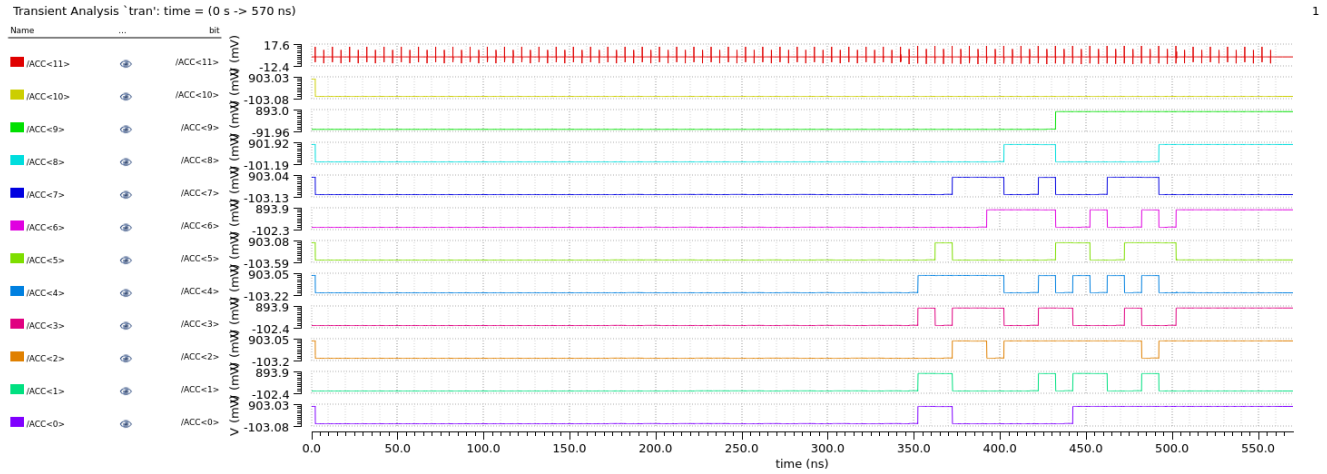


Figure 45: Test A ($T = 10$ ns), ACC⟨11:0⟩: START edge at 332 ns; first accumulate at 352 ns (0x01B = 27), then 51, 156, 156, 216, 324, 324, 478, 556, 567, 583, 661, 685, 755, 805 and **0x34D** = 845 at 502 ns, when DONE rises (17 edges after START); ACC holds until the next START or RST.

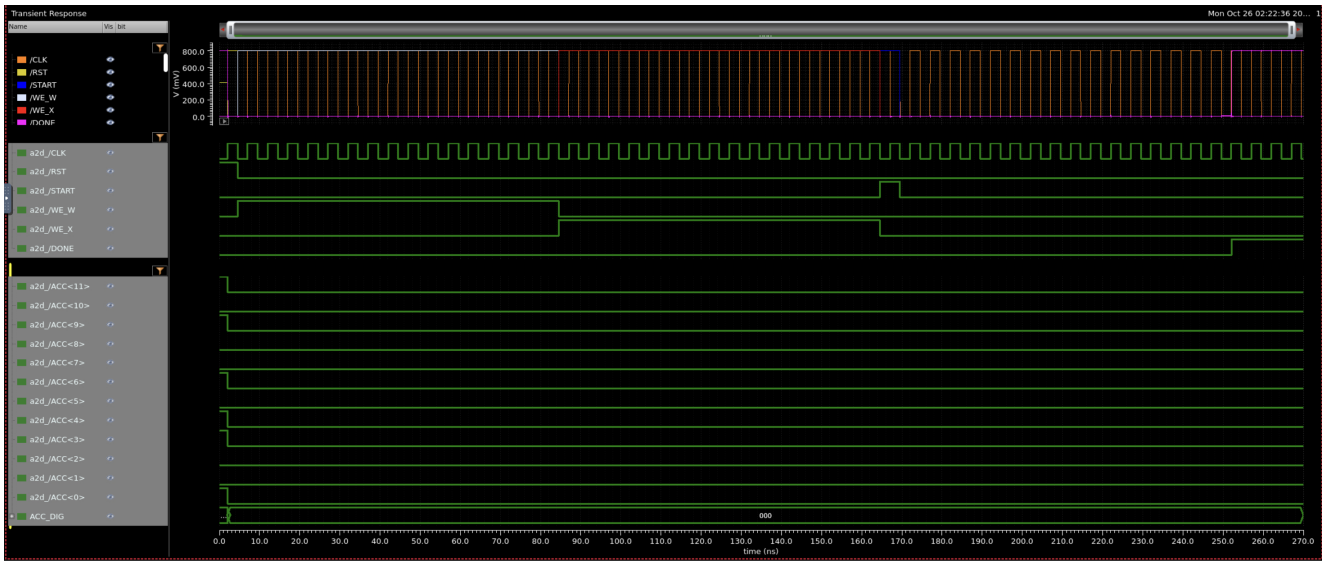


Figure 46: Earlier Test A run, before the read-latch fix: the sequencer finishes (DONE) but ACC stays 0x000.

Read-latch fix found by `tb_top`. The first runs read ACC = 0 although the sequencer was correct (DONE 17 edges after START). Probing inside `top`: WE, ADDR and DIN reach both macros correctly, the wordlines fire and the read inverter output RD rises after each read, but both DOUT buses stay at 0. The macro's output latch was enabled by a pulse derived from the delayed wordline enable (RD_EN = WLD·$\overline{\text{WE_L}}$, about 150 ps wide), which closed before RD had crossed mid-rail, so the latch node peaked near 0.4 V and fell back. RD_EN is now level-sensitive on the clock-high phase (RD_EN = CLKBUF·$\overline{\text{WE_L}}$, one input of `I11` in `controller`): the latch follows RD for the whole high phase and closes about 40 ps before precharge pulls RD down after the falling edge. **[TODO: Rerun Part 2 `tb_sram_macro` with the changed controller.]**

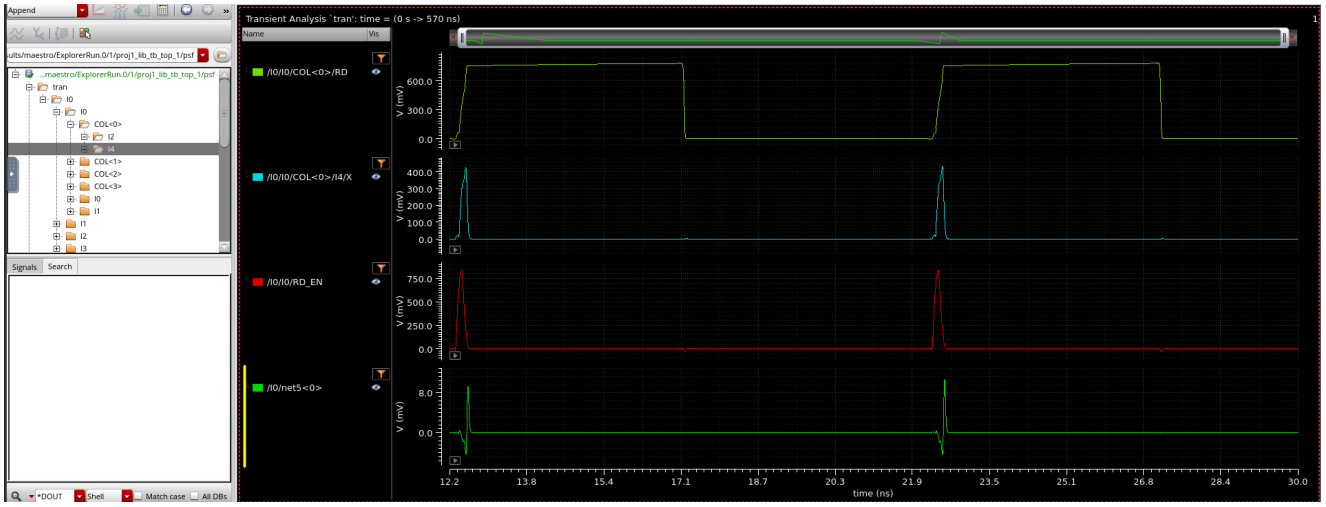


Figure 47: Before the fix: RD rises after the read, but the RD_EN pulse closes first; the latch node X peaks near 0.4 V and returns to 0, so DOUT stays 0.

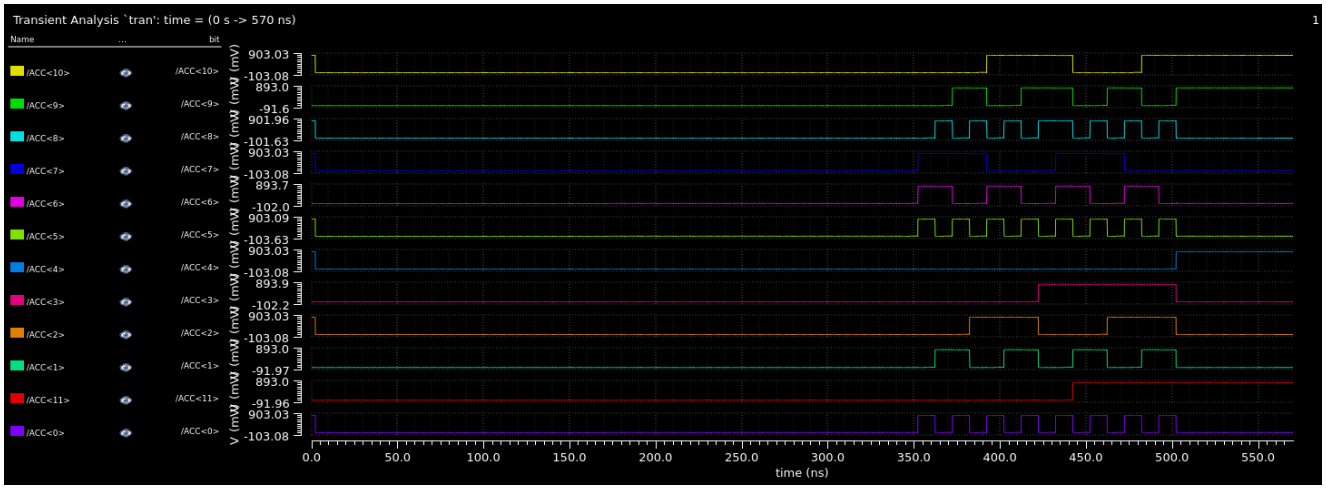


Figure 48: Test B after the fix ($T = 10$ ns): ACC$\langle 11{:}0\rangle$ steps by 225 from 352 ns (0x0E1, 0x1C2, ...) and reaches 0xE10 = 3600 at 502 ns, when DONE rises; ACC holds afterwards.

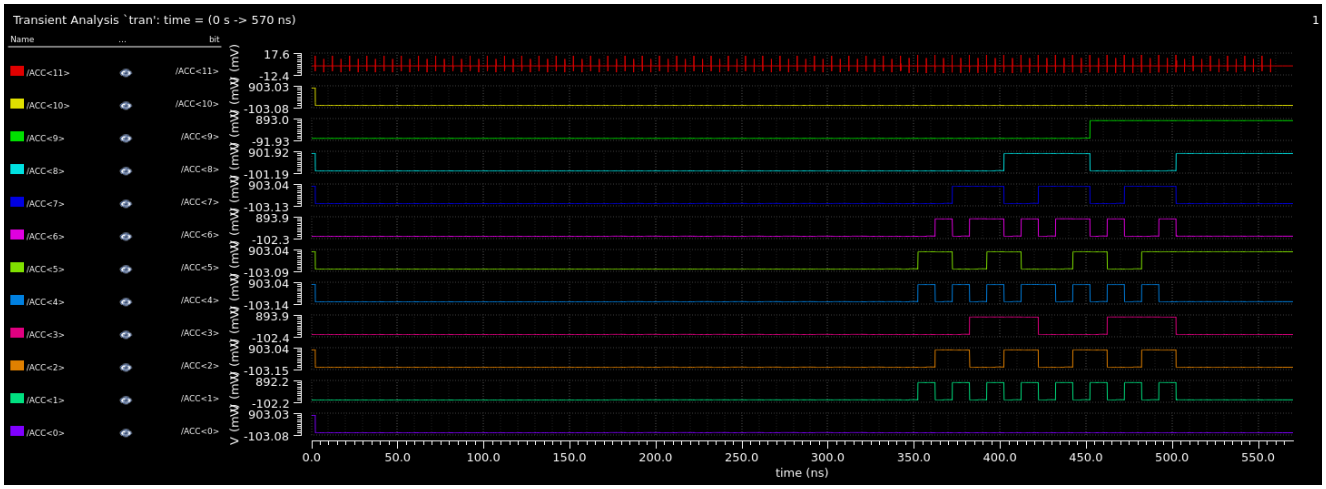


Figure 49: Test C ($T = 10$ ns): ACC steps by 50 from 352 ns (0x032, 0x064, ...) to 0x320 = 800 at 502 ns. The memory outputs alternate 0101/1010 every cycle, the maximum-toggling case.

[missing figure: tb_top_testA_partial_sums]

Figure 50: [TODO: Optional: running partial sums against the hand-computed values for Test A, once ACC is non-zero.]

[TODO: Also check, if time allows: RST mid-computation, START during compute, back-to-back dot products without reloading (ACC clears on START).]

7. Metrics and figure of merit

Rubric (handout): $T_{CLK,\min}$ and $t_{dot} = N_{cycles} \cdot T_{CLK,\min}$ (START to DONE); E_{dot} = supply energy from START to DONE at $T_{CLK,\min}$ for Test A, also B and C, split into memory, datapath, sequencer if separate supplies; area = 2 macros (Part 2) + datapath and sequencer with D; FOM_{MAC} = Area $\cdot E_{dot} \cdot t_{dot}$ (μm^2, pJ, ns); energy per MAC with and without the memory accesses; paragraph on where the energy goes.

At $V_{DD} = 0.8$ V, nominal, 27°C, $D = 28$:

Metric	Value
$T_{CLK,\min}$	**[TODO: ns]** (measured); hand estimate **[TODO: ns]**
N_{cycles} (START to DONE)	17 (predicted) **[TODO: confirm]**
$t_{dot} = N_{cycles}\, T_{CLK,\min}$	**[TODO: ns]**
Area: 2 macros + datapath + sequencer	2×54.6 *(est.)* $+ D \sum W_i L_i$ (7544 fins $\times$ 0.0242 μm^2: datapath 5696 fins = 137.8, sequencer and glue 1848 fins = 44.7) = **291.8 μm^2** *(est.)*
FOM$_{MAC}$ = **Area**$\cdot E_{dot} \cdot t_{dot}$ (Test A)	**[TODO: μm^2·pJ·ns]**

Energy (integral of supply current $\times V_{DD}$ from START to DONE, at $T_{CLK,\min}$):

Test	E_{dot} total	Memory (2 macros)	Datapath	Sequencer
A	**[TODO: pJ]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
B (all 15)	**[TODO: pJ]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
C (5/10 alternating)	**[TODO: pJ]**	**[TODO:]**	**[TODO:]**	**[TODO:]**

[TODO: State how the split is obtained (separate supply sources per block, with the registers counted in the block they belong to).]

	With memory accesses	Without memory accesses
Energy per MAC, Test A ($E_{dot}/16$)	**[TODO: pJ]**	**[TODO: pJ]**

Where the energy goes. [TODO: Paragraph: share of memory against datapath and sequencer; clock tree and register energy per clock (§2) times the number of registers; why B and C differ from A (toggling); static versus dynamic.]

8. What limited the design

The clock period is set by the multiply-accumulate path, not the memory: the array multiplier alone takes about 1.2 ns in its worst case, over twice the 488 ps SRAM read, before the 12-bit ripple adder and the register setup are added. Both arithmetic blocks are ripple structures built from one small full adder (2-fin `xor2` stacks with input inverters), which made them easy to verify but long. In area the datapath (138 μm^2) is larger than the two macros (109 μm^2 *(est.)*), mostly the 24 full adders and the 12 accumulator registers. **[TODO: Energy split once measured.]** With more time: a carry-save multiplier, a faster final adder (carry-select or lookahead), upsized `xor2` stacks, and a pipeline register after the multiplier.

- **Register style:** static TG master-slave, 100 fins per flip-flop (152 with enable and reset), reused everywhere; a dynamic latch would be smaller and faster but cannot hold with the clock stopped. **[TODO: t_{cq}, t_{su}, energy per clock.]**
- **Pipelined vs. not:** overlapping the read with the multiply-accumulate costs one cycle (17 instead of 16) and stays within the 20-cycle limit; because the macro output latch is the only pipeline stage, the SRAM access and the arithmetic still share one period. A register after the multiplier would split them, at a second extra cycle (18).
- **Multiplier and adder architecture:** ripple array and ripple adder: smallest design effort and regular, but the slowest choice; a carry-save tree and a carry-select final adder are the first changes for speed.